

Table of Contents

Scheduling Algorithm.....	1
1. WAP to implement First Come First Serve (FCFS) algorithm.....	1
2. WAP to implement Shortest Job First (SJF) algorithm.	4
3. WAP to implement Shortest Remaining Time Next algorithm.....	7
4. WAP to implement Round Robin algorithm.	9
5. WAP to implement Highest Response Ratio Next (HRRN) algorithm.	12
6. WAP to implement non-preemptive priority scheduling algorithm.....	15
7. WAP to implement preemptive priority scheduling algorithm.	18
Page Replacement Algorithm	21
1. WAP to implement First in First Out page replacement algorithm.....	21
2. WAP to implement second chance page replacement algorithm.	23
3. WAP to implement optimal page replacement algorithm.	25
4. WAP to implement Least Recently Used algorithm.....	27
5. WAP to implement Least Frequently Used algorithm.....	29
6. WAP to implement Clock Page replacement algorithm.	31
Disk Scheduling Algorithm.....	33
1. WAP to implement First Come First Serve algorithm.	33
2. WAP to implement Shortest Seek Time First algorithm.	34
3. WAP to implement Scan Disk Scheduling Algorithm.....	36
4. WAP to implement C-Scan Disk Scheduling Algorithm.	38
5. WAP to implement Look Disk Scheduling Algorithm.	40
6. WAP to implement C-Look disk scheduling algorithm.....	42
Memory Allocation Algorithm.....	44
1. WAP to implement First Fit Memory Allocation Algorithm.	44
2. WAP to implement Best Fit Memory Allocation Algorithm.	46
3. WAP to implement Worst Fit Memory Allocation Algorithm.	48
Deadlocks	50
1. WAP to implement Bankers Algorithm.	50
2. WAP to implement deadlock detection algorithm.	53
3. WAP to implement deadlock recovery.....	56
Classical IPC Problems	61
1. WAP to implement Dining Philosopher Problem Algorithm.....	61
2. WAP to implement Readers and Writes Problem Algorithm.	64
3. WAP to implement Sleep Barbers problem algorithm.....	66

1. WAP to implement producer-consumer problem.	68
Process Creation	70
1. WAP to create Parent or child process.....	70
2. WAP to create orphan process.....	71
3. WAP to create a demon Process	72

Scheduling Algorithm

1. WAP to implement First Come First Serve (FCFS) algorithm.

Source Code:

```
import java.util.Scanner;

public class OSFcFsScheduling {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the total number of processes:");

        int PuraProcesses = sc.nextInt();

        int[] processId = new int[PuraProcesses];
        int[] burstTime = new int[PuraProcesses];
        int[] releaseTime = new int[PuraProcesses];
        int[] completionTime = new int[PuraProcesses];
        int[] turnAroundTime = new int[PuraProcesses];
        int[] waitingTime = new int[PuraProcesses];

        for (int p = 0; p < PuraProcesses; p++) {

            System.out.println("Enter process" + (p + 1) + " Burst Time:");

            burstTime[p] = sc.nextInt();

            System.out.println("Enter process" + (p + 1) + " Release Time:");

            releaseTime[p] = sc.nextInt();

            processId[p] = p + 1;

        }

        int asthayi;

        for (int outerLoop = 0; outerLoop < PuraProcesses; outerLoop++) {

            for (int innerLoop = outerLoop + 1; innerLoop < PuraProcesses; innerLoop++) {

                if (releaseTime[outerLoop] > releaseTime[innerLoop]) {

                    asthayi = releaseTime[outerLoop];

                    releaseTime[outerLoop] = releaseTime[innerLoop];

                    releaseTime[innerLoop] = asthayi;

                }

            }

        }

    }

}
```

```

        asthayi = processId[outerLoop];
        processId[outerLoop] = processId[innerLoop];
        processId[innerLoop] = asthayi;

        asthayi = burstTime[outerLoop];
        burstTime[outerLoop] = burstTime[innerLoop];
        burstTime[innerLoop] = asthayi;
    }
}

int currentTime = 0;
for (int i = 0; i < PuraProcesses; i++) {
    if (currentTime < releaseTime[i]) {
        // CPU is idle until process arrives
        currentTime = releaseTime[i];
    }
    currentTime += burstTime[i];
    completionTime[i] = currentTime;
    turnAroundTime[i] = completionTime[i] - releaseTime[i];
    waitingTime[i] = turnAroundTime[i] - burstTime[i];
}

System.out.println("Process ID\tRelease Time\tBurst Time\tCompletion Time\tTurn
Around Time\tWaiting Time");

for (int n = 0; n < PuraProcesses; n++) {
    System.out.println(processId[n] + "\t\t" + releaseTime[n] + "\t\t" + burstTime[n] +
"\t\t"
        + completionTime[n] + "\t\t" + turnAroundTime[n] + "\t\t\t" + waitingTime[n]);
}

System.out.println("\nGantt Chart:");
int samaye = 0;

```

```

for (int i = 0; i < PuraProcesses; i++) {
    if (samaye < releaseTime[i]) {
        System.out.print("| Idle ");
        samaye = releaseTime[i];
    }
    System.out.print("| P" + processId[i] + " ");
    samaye += burstTime[i];
}
System.out.println("");
}
}

```

Output

```

E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs>java OSFcFsScheduling.java
Enter the total number of processes:
4
Enter process1 Brust Time:
5
Enter process1 Release Time:
3
Enter process2 Brust Time:
8
Enter process2 Release Time:
2
Enter process3 Brust Time:
10
Enter process3 Release Time:
5
Enter process4 Brust Time:
3
Enter process4 Release Time:
12

```

Process ID	Release Time	Brust Time	Completion Time	Turn Around Time	Waiting Time
2	2	8	10	8	0
1	3	5	15	12	7
3	5	10	25	20	10
4	12	3	28	16	13

```

Gantt Chart:
| Idle | P2 | P1 | P3 | P4 |
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs>

```

2. WAP to implement Shortest Job First (SJF) algorithm.

Source code:

```
import java.util.Scanner;

public class SJFOSBG {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the total number of processes:");

        int PuraProcesses = sc.nextInt();

        int[] processId = new int[PuraProcesses];
        int[] burstTime = new int[PuraProcesses];
        int[] releaseTime = new int[PuraProcesses];
        int[] completionTime = new int[PuraProcesses];
        int[] turnAroundTime = new int[PuraProcesses];
        int[] waitingTime = new int[PuraProcesses];
        int[] processStatus = new int[PuraProcesses];

        int systemTime = 0, finished = 0;

        float avgwt = 0, avgta = 0;

        for (int p = 0; p < PuraProcesses; p++) {

            System.out.println("Enter process" + (p + 1) + " Burst Time:");

            burstTime[p] = sc.nextInt();

            System.out.println("Enter process" + (p + 1) + " Release Time:");

            releaseTime[p] = sc.nextInt();

            processId[p] = p + 1;

            processStatus[p] = 0;

        }

        while (true) {

            int c = PuraProcesses, min = 9999;

            if (finished == PuraProcesses)

                break;
```

```

    for (int i = 0; i < PuraProcesses; i++) {
        if ((releaseTime[i] <= systemTime) && (processStatus[i] == 0) &&
(burstTime[i] < min)) {
            min = burstTime[i];
            c = i;
        }
    }
    if (c == PuraProcesses)
        systemTime++;
    else {
        completionTime[c] = systemTime + burstTime[c];
        systemTime += burstTime[c];
        turnAroundTime[c] = completionTime[c] - releaseTime[c];
        waitingTime[c] = turnAroundTime[c] - burstTime[c];
        processStatus[c] = 1;
        processId[finished] = c + 1;
        finished++;
    }
}

    System.out.println("Process ID\tRelease Time\tBurst Time\tCompletion
Time\tTurn Around Time\tWaiting Time");
    for (int n = 0; n < PuraProcesses; n++) {
        avgwt += waitingTime[n];
        avgta += turnAroundTime[n];
        int idx = processId[n] - 1; // FIX
        System.out.println(processId[n] + "\t\t" + releaseTime[idx] + "\t\t" +
            burstTime[idx] + "\t\t" + completionTime[idx] + "\t\t" +
            turnAroundTime[idx] + "\t\t\t" + waitingTime[idx]);
    }

```

```

        System.out.println("\n Average Turn Around Time: " + (float) (avgta /
PuraProcesses));

        System.out.println("\n Average Waiting Time: " + (float) (avgwt / PuraProcesses));

        sc.close();

        for (int i = 0; i < PuraProcesses; i++) {

            System.out.print("| P" + processId[i] + " ");

        }

    }

}

```

Output:

```

C:\Windows\System32\cmd.exe
E:\Bca-A11-Notes\4 Fourth Semester\Operating System\Algorithm and Programs>java SJFOSBG.java
Enter the total number of processes:
5
Enter process1 Brust Time:
6
Enter process1 Release Time:
3
Enter process2 Brust Time:
7
Enter process2 Release Time:
5
Enter process3 Brust Time:
0
Enter process3 Release Time:
9
Enter process4 Brust Time:
9
Enter process4 Release Time:
10
Enter process5 Brust Time:
6
Enter process5 Release Time:
7
Process ID      Release Time    Brust Time      Completion Time  Turn Around Time    Waiting Time
1               3               6               9                6                   0
3               9               0               9                0                   0
5               7               6               15               8                   2
2               5               7               22               17                  10
4               10              9               31               21                  12

Average Turn Around Time: 10.4

Average Waiting Time: 4.8
| P1 | P3 | P5 | P2 | P4
E:\Bca-A11-Notes\4 Fourth Semester\Operating System\Algorithm and Programs>

```


3. WAP to implement Shortest Remaining Time Next algorithm.

Source Code:

```
import java.util.Scanner;

public class STRNOSBG {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the total number of processes:");

        int PuraProcesses = sc.nextInt();

        int[] processId = new int[PuraProcesses];
        int[] burstTime = new int[PuraProcesses];
        int[] releaseTime = new int[PuraProcesses];
        int[] completionTime = new int[PuraProcesses];
        int[] turnAroundTime = new int[PuraProcesses];
        int[] waitingTime = new int[PuraProcesses];
        int[] remainingTime = new int[PuraProcesses];
        int[] processStatus = new int[PuraProcesses];

        float avgwt = 0, avgta = 0;

        for (int p = 0; p < PuraProcesses; p++) {

            System.out.println("Enter process" + (p + 1) + " Burst Time:");

            burstTime[p] = sc.nextInt();

            System.out.println("Enter process" + (p + 1) + " Release Time:");

            releaseTime[p] = sc.nextInt(); processId[p] = p + 1;

            remainingTime[p] = burstTime[p]; }

        int systemTime = 0, completeProcess = 0;

        while (completeProcess < PuraProcesses) {

            int shortestKamIndex = -1; int min = 9999;

            for (int i = 0; i < PuraProcesses; i++) {

                if ((releaseTime[i] <= systemTime) && remainingTime[i] > 0 &&
remainingTime[i] < min) {

                    min = remainingTime[i]; shortestKamIndex = i;}}

            systemTime = systemTime + min;

            completeProcess++;

            remainingTime[shortestKamIndex] = remainingTime[shortestKamIndex] - min;

            if (remainingTime[shortestKamIndex] == 0) {

                completionTime[shortestKamIndex] = systemTime;

                turnAroundTime[shortestKamIndex] = completionTime[shortestKamIndex] - releaseTime[shortestKamIndex];

                waitingTime[shortestKamIndex] = turnAroundTime[shortestKamIndex];

                avgwt = (avgwt * completeProcess + waitingTime[shortestKamIndex]) / (completeProcess + 1);

                avgta = (avgta * completeProcess + turnAroundTime[shortestKamIndex]) / (completeProcess + 1);

                completeProcess++;

            }

        }

        System.out.println("Average waiting time is: " + avgwt);

        System.out.println("Average turn around time is: " + avgta);

    }

}
```

```

        if (shortestKamIndex == -1) { systemTime++; continue; }

        remainingTime[shortestKamIndex]--; systemTime++;

        if (remainingTime[shortestKamIndex] == 0) {

            completeProcess++; processStatus[shortestKamIndex] = 1;

            completionTime[shortestKamIndex] = systemTime;

            turnAroundTime[shortestKamIndex] = completionTime[shortestKamIndex] -
            - releaseTime[shortestKamIndex];

            waitingTime[shortestKamIndex] = turnAroundTime[shortestKamIndex] -
            burstTime[shortestKamIndex];

            avgta += turnAroundTime[shortestKamIndex];

            avgwt += waitingTime[shortestKamIndex];

        } }

        System.out.println("Process ID\tRelease Time\tBurst Time\tCompletion
        Time\tTurn Around Time\tWaiting Time");

        for (int n = 0; n < PuraProcesses; n++) {

            System.out.println(processId[n] + "\t\t" + releaseTime[n] + "\t\t" +

                burstTime[n] + "\t\t" + completionTime[n] + "\t\t" +

                turnAroundTime[n] + "\t\t" + waitingTime[n]); }

        System.out.println("\nAverage Turn Around Time: " + (avgta / PuraProcesses));

        System.out.println("Average Waiting Time: " + (avgwt / PuraProcesses));

        sc.close();}}

```

Output:

```

Select C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs>java STRNOSBG.java
Enter the total number of processes:
3
Enter process1 Burst Time:
6
Enter process1 Release Time:
7
Enter process2 Burst Time:
9
Enter process2 Release Time:
2
Enter process3 Burst Time:
11
Enter process3 Release Time:
10
Process ID      Release Time    Burst Time      Completion Time  Turn Around Time  Waiting Time
1               7               6               17               10               4
2               2               9               11               9                0
3               10              11              28               18               7
Average Turn Around Time: 12.33333
Average Waiting Time: 3.666667
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs>

```

4. WAP to implement Round Robin algorithm.

Source Code:

```
import java.util.Scanner;

public class RRobinBG {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the total number of processes:");
        int PuraProcesses = sc.nextInt();

        System.out.println("Enter Quantum Time");
        int Quantum = sc.nextInt();

        int[] processId = new int[PuraProcesses];
        int[] burstTime = new int[PuraProcesses];
        int[] releaseTime = new int[PuraProcesses];
        int[] completionTime = new int[PuraProcesses];
        int[] remainingTime = new int[PuraProcesses];

        for (int p = 0; p < PuraProcesses; p++) {

            System.out.println("Enter process" + (p + 1) + " Burst Time:");
            burstTime[p] = sc.nextInt();

            System.out.println("Enter process" + (p + 1) + " Release Time:");
            releaseTime[p] = sc.nextInt();

            processId[p] = p + 1;
            remainingTime[p] = burstTime[p];

        }

        int systemTime = 0, completeProcess = 0;
        float totalTT = 0, totalWT = 0;
        while (completeProcess < PuraProcesses) {

            boolean sakiyo = false;
```

```

        for (int i = 0; i < PuraProcesses; i++) {
            if (remainingTime[i] > 0 && releaseTime[i] <= systemTime)
            {
                sakiyo = true;
                if (remainingTime[i] <= Quantum) {
                    systemTime += remainingTime[i];
                    remainingTime[i] = 0;
                    completionTime[i] = systemTime;
                    completeProcess++;
                } else {
                    systemTime += Quantum;
                    remainingTime[i] -= Quantum;
                }
            }
            if (!sakiyo) {
                systemTime++;
            }
        }
    }

```

```

        System.out.println("Process ID\tRelease Time\tBurst Time\tCompletion Time\tTurn Around Time\tWaiting Time");

```

```

        for (int n = 0; n < PuraProcesses; n++) {
            int turnAroundTime = completionTime[n] - releaseTime[n];
            int waitingTime = turnAroundTime - burstTime[n];
            totalTT += turnAroundTime;
            totalWT += waitingTime;
            System.out.println(processId[n] + "\t\t" + releaseTime[n] + "\t\t" + burstTime[n] + "\t\t" + completionTime[n] + "\t\t" + turnAroundTime + "\t\t" + waitingTime);
        }
    }

```

```

    }

    System.out.println("\nAverage Turn Around Time: " + (totalTT /
PuraProcesses));

    System.out.println("Average Waiting Time: " + (totalWT / PuraProcesses));

    sc.close();

}

}

```

Output:

```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs>java RRobinBG.java
Enter the total number of processes:
4
Enter Quantum Time
2
Enter process1 Burst Time:
6
Enter process1 Release Time:
5
Enter process2 Burst Time:
7
Enter process2 Release Time:
0
Enter process3 Burst Time:
8
Enter process3 Release Time:
2
Enter process4 Burst Time:
9
Enter process4 Release Time:
3
Process ID    Release Time    Burst Time    Completion Time    Turn Around Time    Waiting Time
1             5               6             24                 19                  13
2             0               7             25                 25                  18
3             2               8             27                 25                  17
4             3               9             30                 27                  18

Average Turn Around Time: 24.0
Average Waiting Time: 16.5

E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs>

```

5. WAP to implement Highest Response Ratio Next (HRRN) algorithm.

Source Code:

```
import java.util.Scanner;

public class HRRNBG {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the total number of processes:");

        int PuraProcesses = sc.nextInt();

        int[] processId = new int[PuraProcesses];
        int[] burstTime = new int[PuraProcesses];
        int[] releaseTime = new int[PuraProcesses];
        int[] completionTime = new int[PuraProcesses];
        int[] turnAroundTime = new int[PuraProcesses];
        int[] waitingTime = new int[PuraProcesses];
        boolean[] sakiyo = new boolean[PuraProcesses];

        for (int p = 0; p < PuraProcesses; p++) {

            System.out.println("Enter process" + (p + 1) + " Burst Time:");

            burstTime[p] = sc.nextInt();

            System.out.println("Enter process" + (p + 1) + " Release Time:");

            releaseTime[p] = sc.nextInt();

            processId[p] = p + 1;

        }

        int systemTime = 0, completeProcess = 0;

        float totalTT = 0, totalWT = 0;

        while (completeProcess < PuraProcesses) {

            int thuloResponseIndex = -1;

            float maxResponseRatio = -1;

            for (int i = 0; i < PuraProcesses; i++) {
```

```

        if (!sakiyo[i] && releaseTime[i] <= systemTime) {
            float responseRatio = (float) (systemTime - releaseTime[i] + burstTime[i]) /
burstTime[i];
            if (responseRatio > maxResponseRatio) {
                maxResponseRatio = responseRatio;
                thuloResponseIndex = i;
            } else if (responseRatio == maxResponseRatio && thuloResponseIndex != -
1) {
                if (releaseTime[i] < releaseTime[thuloResponseIndex]) {
                    thuloResponseIndex = i;
                }
            }
        }
        if (thuloResponseIndex == -1) {
            systemTime++;
            continue;
        } else {
            systemTime += burstTime[thuloResponseIndex];
            completionTime[thuloResponseIndex] = systemTime;
            turnAroundTime[thuloResponseIndex] = completionTime[thuloResponseIndex]
                - releaseTime[thuloResponseIndex];
            waitingTime[thuloResponseIndex] = turnAroundTime[thuloResponseIndex] -
burstTime[thuloResponseIndex];
            totalTT += turnAroundTime[thuloResponseIndex];
            totalWT += waitingTime[thuloResponseIndex];
            sakiyo[thuloResponseIndex] = true;
            completeProcess++;
        }
    }

    System.out.println("Process ID\tRelease Time\tBurst Time\tCompletion Time\tTurn
Around Time\tWaiting Time");

```

```

for (int n = 0; n < PuraProcesses; n++) {

    System.out.println(processId[n] + "\t\t" + releaseTime[n] + "\t\t" +
        burstTime[n] + "\t\t" + completionTime[n] + "\t\t" +
        turnAroundTime[n] + "\t\t\t" + waitingTime[n]);

}

System.out.println("\nAverage Turn Around Time: " + (totalTT / PuraProcesses));

System.out.println("Average Waiting Time: " + (totalWT / PuraProcesses));

sc.close();

}

}

```

Output:

```

C:\Windows\System32\cmd.exe

E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs>java HRRNBG.java
Enter the total number of processes:
6
Enter process1 Burst Time:
5
Enter process1 Release Time:
4
Enter process2 Burst Time:
7
Enter process2 Release Time:
3
Enter process3 Burst Time:
9
Enter process3 Release Time:
0
Enter process4 Burst Time:
5
Enter process4 Release Time:
3
Enter process5 Burst Time:
8
Enter process5 Release Time:
8
Enter process6 Burst Time:
9
Enter process6 Release Time:
9

```

Process ID	Release Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
1	4	5	19	15	10
2	3	7	26	23	16
3	0	9	9	9	0
4	3	5	14	11	6
5	8	8	34	26	18
6	9	9	43	34	25

```

Average Turn Around Time: 19.666666
Average Waiting Time: 12.5

```


6. WAP to implement non-preemptive priority scheduling algorithm.

Source Code:

```
import java.util.Scanner;

public class NPPBG {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the total number of processes:");

        int PuraProcesses = sc.nextInt();

        int[] processId = new int[PuraProcesses];

        int[] burstTime = new int[PuraProcesses];

        int[] releaseTime = new int[PuraProcesses];

        int[] completionTime = new int[PuraProcesses];

        int[] turnAroundTime = new int[PuraProcesses];

        int[] waitingTime = new int[PuraProcesses];

        int[] processPriority = new int[PuraProcesses];

        boolean[] sakiyo = new boolean[PuraProcesses];

        for (int p = 0; p < PuraProcesses; p++) {

            System.out.println("Enter process" + (p + 1) + " Brust Time:");

            burstTime[p] = sc.nextInt();

            System.out.println("Enter process" + (p + 1) + " Release Time:");

            releaseTime[p] = sc.nextInt();

            System.out.println("Enter process" + (p + 1) + " Priority:");

            processPriority[p] = sc.nextInt();

            processId[p] = p + 1;

        }

        int systemTime = 0, finished = 0;

        float totalTAT = 0, totalWT = 0;
```

```

while (finished < PuraProcesses) {
    int thuloPriorityIndex = -1;
    int thuloPriority = 9999;

    for (int i = 0; i < PuraProcesses; i++) {
        if (!sakiyo[i] && releaseTime[i] <= systemTime && processPriority[i] <
thuloPriority) {
            thuloPriority = processPriority[i];
            thuloPriorityIndex = i;
        }
    }
    if (thuloPriorityIndex == -1) {
        systemTime++;
        continue;
    }

    systemTime += burstTime[thuloPriorityIndex];
    completionTime[thuloPriorityIndex] = systemTime;
    turnAroundTime[thuloPriorityIndex] = completionTime[thuloPriorityIndex] -
releaseTime[thuloPriorityIndex];
    waitingTime[thuloPriorityIndex] = turnAroundTime[thuloPriorityIndex] -
burstTime[thuloPriorityIndex];

    totalTAT += turnAroundTime[thuloPriorityIndex];
    totalWT += waitingTime[thuloPriorityIndex];
    sakiyo[thuloPriorityIndex] = true;
    finished++;
}

```

```

System.out.println(

        "Process ID\tRelease Time\tBurst Time\tPriority\tCompletion Time\tTurn
Around Time\tWaiting Time");

    for (int n = 0; n < PuraProcesses; n++) {

        System.out.println(processId[n] + "\t\t" + releaseTime[n] + "\t\t" +

            burstTime[n] + "\t\t" + processPriority[n] + "\t\t" + completionTime[n] +

            "\t\t" +

                turnAroundTime[n] + "\t\t\t" + waitingTime[n]);

    }

    System.out.println("\nAverage Turn Around Time: " + (totalTAT / PuraProcesses));

    System.out.println("Average Waiting Time: " + (totalWT / PuraProcesses));

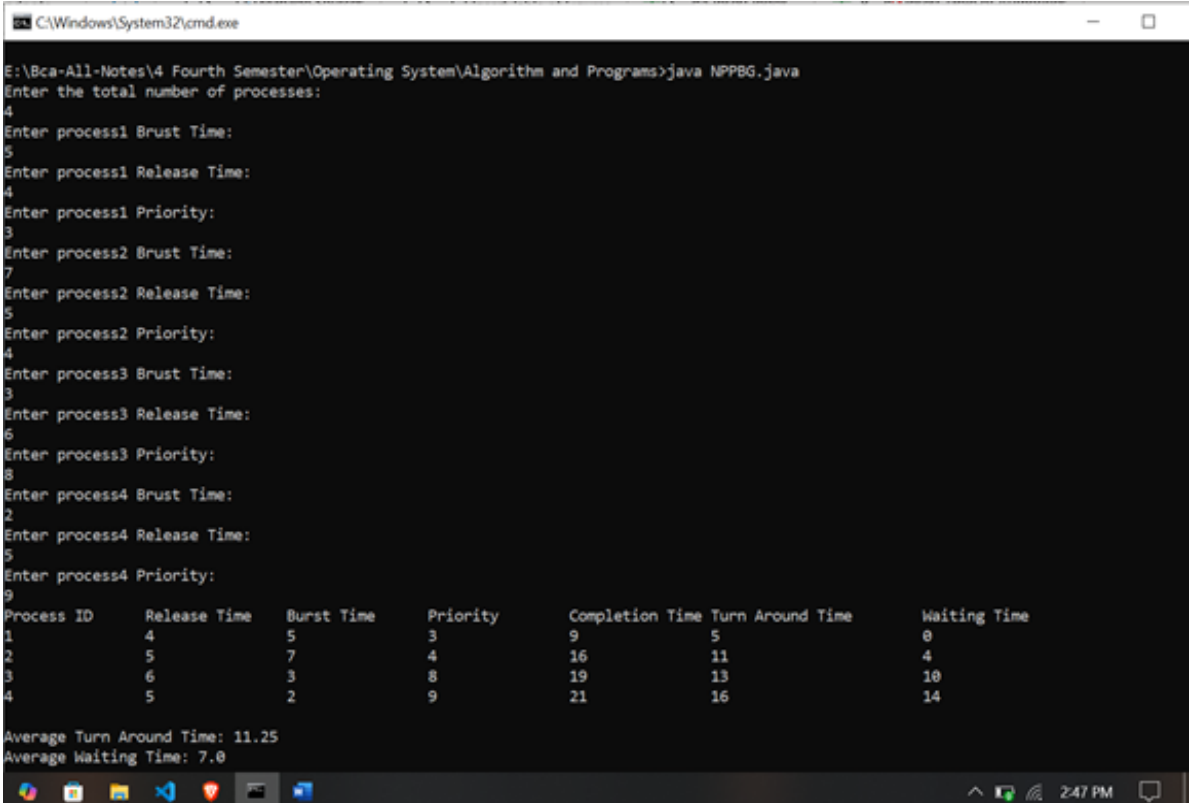
    sc.close();

}

}

```

Output:



```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs>java NPPBG.java
Enter the total number of processes:
4
Enter process1 Brust Time:
5
Enter process1 Release Time:
4
Enter process1 Priority:
3
Enter process2 Brust Time:
7
Enter process2 Release Time:
5
Enter process2 Priority:
4
Enter process3 Brust Time:
3
Enter process3 Release Time:
6
Enter process3 Priority:
8
Enter process4 Brust Time:
2
Enter process4 Release Time:
5
Enter process4 Priority:
9
Process ID    Release Time    Burst Time    Priority    Completion Time    Turn Around Time    Waiting Time
1             4               5             3           9                  5                   0
2             5               7             4           16                 11                  4
3             6               3             8           19                 13                  10
4             5               2             9           21                 16                  14
Average Turn Around Time: 11.25
Average Waiting Time: 7.0

```

7. WAP to implement preemptive priority scheduling algorithm.

Source code:

```
import java.util.Scanner;

public class PPSBG {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the total number of processes:");

        int PuraProcesses = sc.nextInt();

        int[] processId = new int[PuraProcesses];
        int[] burstTime = new int[PuraProcesses];
        int[] releaseTime = new int[PuraProcesses];
        int[] completionTime = new int[PuraProcesses];
        int[] turnAroundTime = new int[PuraProcesses];
        int[] waitingTime = new int[PuraProcesses];
        int[] remaningTime = new int[PuraProcesses];
        int[] processPriority = new int[PuraProcesses];
        boolean[] sakiyo = new boolean[PuraProcesses];

        for (int p = 0; p < PuraProcesses; p++) {

            System.out.println("Enter process" + (p + 1) + " Brust Time:");

            burstTime[p] = sc.nextInt();

            remaningTime[p] = burstTime[p];

            System.out.println("Enter process" + (p + 1) + " Release Time:");

            releaseTime[p] = sc.nextInt();

            System.out.println("Enter process" + (p + 1) + " Priority:");

            processPriority[p] = sc.nextInt();

            processId[p] = p + 1;

        }

        int systemTime = 0, finished = 0;

        float totalTAT = 0, totalWT = 0;
```

```

while (finished < PuraProcesses) {
    int thuloPriorityIndex = -1;
    int thuloPriority = 9999;
    for (int i = 0; i < PuraProcesses; i++) {
        if (!sakiyo[i] && releaseTime[i] <= systemTime && processPriority[i] <
thuloPriority
            && remaningTime[i] > 0) {
            thuloPriority = processPriority[i];
            thuloPriorityIndex = i;
        }
    }
    if (thuloPriorityIndex == -1) {
        systemTime++;
        continue;
    }
    remaningTime[thuloPriorityIndex]--;
    systemTime++;
    if (remaningTime[thuloPriorityIndex] == 0) {
        completionTime[thuloPriorityIndex] = systemTime;
        turnAroundTime[thuloPriorityIndex] = completionTime[thuloPriorityIndex]
            - releaseTime[thuloPriorityIndex];
        waitingTime[thuloPriorityIndex] = turnAroundTime[thuloPriorityIndex] -
burstTime[thuloPriorityIndex];
        totalTAT += turnAroundTime[thuloPriorityIndex];
        totalWT += waitingTime[thuloPriorityIndex];
        sakiyo[thuloPriorityIndex] = true;
        finished++;
    }
}
System.out.println(

```

```

        "Process ID\tRelease Time\tBurst Time\tPriority\tCompletion Time\tTurn Around
Time\tWaiting Time");
    for (int n = 0; n < PuraProcesses; n++) {
        System.out.println(processId[n] + "\t\t" + releaseTime[n] + "\t\t" +
            burstTime[n] + "\t\t" + processPriority[n] + "\t\t" + completionTime[n] + "\t\t\t"
+
            turnAroundTime[n] + "\t\t\t" + waitingTime[n]);
    }
    System.out.println("\nAverage Turn Around Time: " + (totalTAT / PuraProcesses));
    System.out.println("Average Waiting Time: " + (totalWT / PuraProcesses));
    sc.close();
}
}

```

Output:

```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs>java PPSBG.java
Enter the total number of processes:
4
Enter process1 Brust Time:
7
Enter process1 Release Time:
0
Enter process1 Priority:
4
Enter process2 Brust Time:
9
Enter process2 Release Time:
1
Enter process2 Priority:
4
Enter process3 Brust Time:
9
Enter process3 Release Time:
3
Enter process3 Priority:
6
Enter process4 Brust Time:
3
Enter process4 Release Time:
8
Enter process4 Priority:
4
Process ID      Release Time    Burst Time     Priority        Completion Time  Turn Around Time    Waiting Time
1               0               7              4              7               7                  0
2               1               9              4              16              15                 6
3               3               9              6              28              25                 16
4               8               3              4              19              11                 8

Average Turn Around Time: 14.5
Average Waiting Time: 7.5
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs>

```

Page Replacement Algorithm

1. WAP to implement First in First Out page replacement algorithm.

Source Code:

```
import java.util.HashSet;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;

public class FIFOPage {

    static int pageFault(int pages[], int puraPages, int frameSize) {

        HashSet<Integer> s = new HashSet<>(frameSize);
        Queue<Integer> indexes = new LinkedList<>();
        int pageFaults = 0;
        for (int i = 0; i < puraPages; i++) {
            if (s.size() < frameSize) {
                if (!s.contains(pages[i])) {
                    s.add(pages[i]);
                    pageFaults++;
                    indexes.add(pages[i]);
                }
            } else {
                if (!s.contains(pages[i])) {
                    int val = indexes.poll();
                    s.remove(val);
                    s.add(pages[i]);
                    indexes.add(pages[i]);
                    pageFaults++;
                }
            }
        }
        for (int page : indexes) {
```

```

        System.out.print(page + " ");
    } System.out.println();
}    return pageFaults;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter the number of pages:");
    int puraPages = sc.nextInt();
    int pages[] = new int[puraPages];
    System.out.println("Enter the pages ");
    for (int p = 0; p < puraPages; p++) {
        pages[p] = sc.nextInt();
    }
    System.out.println("Enter the number of frames:");
    int frameSize = sc.nextInt();
    int totalPageFaults = pageFault(pages, puraPages, frameSize);
    System.out.println("Total Page Faults: " + totalPageFaults);
}
}

```

Output:

```

E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\PageReplacement>java FIFOpage.java
Enter the number of pages:
6
Enter the pages
1
2
3
2
1
5
Enter the number of frames:
3
1 2
1 2 3
1 2 3
1 2 3
2 3 5
Total Page Faults: 4

```


2. WAP to implement second chance page replacement algorithm.

Source Code:

```
import java.util.LinkedList;
import java.util.Scanner;
public class ArkoChance {
    static int pageFault(int pages[], int puraPages, int frameSize) {
        LinkedList<Integer> frames = new LinkedList<>();
        boolean[] reference = new boolean[frameSize];
        int pageFaults = 0, pointer = 0;
        for (int i = 0; i < puraPages; i++) {
            int page = pages[i];
            boolean found = false;
            for (int j = 0; j < frames.size(); j++) {
                if (frames.get(j) == page) {
                    reference[j] = true;
                    found = true;
                    break; } }
            if (!found) {
                pageFaults++;
                while (true) {
                    if (frames.size() < frameSize) {
                        frames.add(page);
                        reference[frames.indexOf(page)] = true;
                        break;
                    } else if (reference[pointer]) {
                        reference[pointer] = false;
                        pointer = (pointer + 1) % frameSize;
                    } else {
                        frames.set(pointer, page);
                        reference[pointer] = true;
                    }
                }
            }
        }
        return pageFaults;
    }
}
```

```

        pointer = (pointer + 1) % frameSize;

        break; } } }

for ( int f : frames) {

    System.out.print(f + " "); }

    System.out.println(); }

return pageFaults;

}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.println("Enter the number of pages:");

    int puraPages = sc.nextInt();

    int pages[] = new int[puraPages];

    System.out.println("Enter the pages ");

    for (int p = 0; p < puraPages; p++) {

        pages[p] = sc.nextInt(); }

    System.out.println("Enter the number of frames:");

    int frameSize = sc.nextInt();

    int totalPageFaults = pageFault(pages, puraPages, frameSize);

    System.out.println("Total Page Faults: " + totalPageFaults);

}}

```

Output:

```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\PageReplacement>java ArkoChance.java
Enter the number of pages:
7
Enter the pages
1
2
3
3
1
5
3
Enter the number of frames:
3
1
1 2
1 2 3
1 2 3
1 2 3
5 2 3
5 2 3
Total Page Faults: 4
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\PageReplacement>

```

3. WAP to implement optimal page replacement algorithm.

Source Code:

```
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.Scanner;
public class OptimalPRA
{
    static int pageFault(int pages[], int puraPages, int frameSize) {
        ArrayList<Integer> frames = new ArrayList<>();
        int pageFaults = 0;
        for (int i = 0; i < puraPages; i++) {
            int page = pages[i];
            if (!frames.contains(page)) {
                pageFaults++;
                if (frames.size() == frameSize) {
                    int tada = -1, replaceIndex = -1;
                    for (int j = 0; j < frames.size(); j++) {
                        int framePage = frames.get(j);
                        int arkoUse = 99999;
                        for (int k = i + 1; k < puraPages; k++) {
                            if (pages[k] == framePage) {
                                arkoUse = k;
                                break;
                            }
                        }
                    }
                    if (arkoUse > tada) {
                        tada = arkoUse;
                        replaceIndex = j;
                    }
                    frames.set(replaceIndex, page);
                } else {
                    frames.add(page);
                }
            }
        }
        for (int f : frames) {
        }
```

```

        System.out.print(f + " ") }

        System.out.println() }

    return pageFaults; }

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.println("Enter the number of pages:");

    int puraPages = sc.nextInt();

    int pages[] = new int[puraPages];

    System.out.println("Enter the pages ");

    for (int p = 0; p < puraPages; p++) {

        pages[p] = sc.nextInt();

    }

    System.out.println("Enter the number of frames:");

    int frameSize = sc.nextInt();

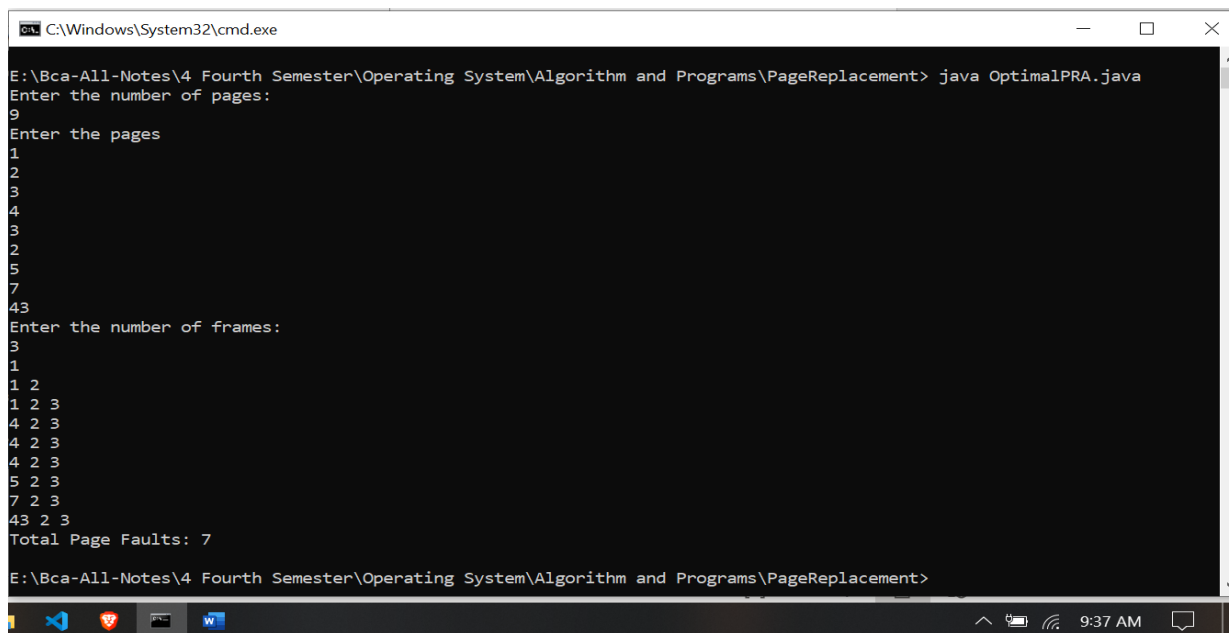
    int totalPageFaults = pageFault(pages, puraPages, frameSize);

    System.out.println("Total Page Faults: " + totalPageFaults);

}}

```

Output:



```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\PageReplacement> java OptimalPRA.java
Enter the number of pages:
9
Enter the pages
1
2
3
4
3
2
5
7
43
Enter the number of frames:
3
1
1 2
1 2 3
4 2 3
4 2 3
4 2 3
5 2 3
7 2 3
43 2 3
Total Page Faults: 7
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\PageReplacement>

```

4. WAP to implement Least Recently Used algorithm.

Source Code:

```
import java.util.LinkedList;
import java.util.Scanner;
public class LRUPage {
    static int pageFault(int pages[], int puraPages, int frameSize) {
        LinkedList<Integer> frames = new LinkedList<>();
        int pageFaults = 0;
        for (int i = 0; i < puraPages; i++) {
            int page = pages[i];
            if (!frames.contains(page)) {
                pageFaults++;
                if (frames.size() < frameSize) {
                    frames.add(page);
                } else {
                    frames.removeFirst();
                    frames.add(page);
                }
            } else {
                frames.remove((Integer) page);
                frames.add(page);
            }
            for (int f : frames) {
                System.out.print(f + " ");
            }
            System.out.println();
        }
        return pageFaults;
    }
    public static void main(String[] args) {
```

```

Scanner sc = new Scanner(System.in);

System.out.println("Enter the number of pages:");

int puraPages = sc.nextInt();

int pages[] = new int[puraPages];

System.out.println("Enter the pages ");

for (int p = 0; p < puraPages; p++) {

    pages[p] = sc.nextInt();

}

System.out.println("Enter the number of frames:");

int frameSize = sc.nextInt();

int totalPageFaults = pageFault(pages, puraPages, frameSize);

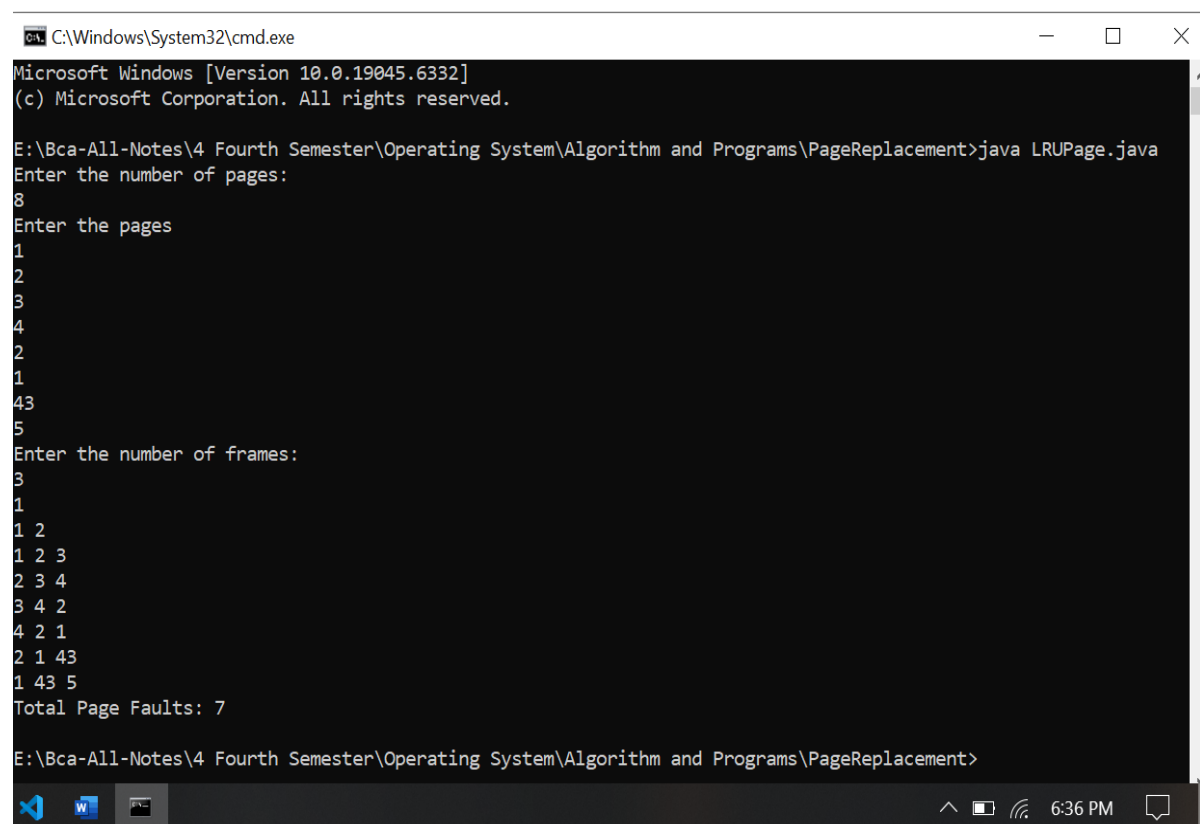
System.out.println("Total Page Faults: " + totalPageFaults);

}

}

```

Output:



```

C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.6332]
(c) Microsoft Corporation. All rights reserved.

E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\PageReplacement>java LRUPage.java
Enter the number of pages:
8
Enter the pages
1
2
3
4
2
1
4
3
Enter the number of frames:
3
1
1 2
1 2 3
2 3 4
3 4 2
4 2 1
2 1 43
1 43 5
Total Page Faults: 7

E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\PageReplacement>

```

5. WAP to implement Least Frequently Used algorithm.

Source Code:

```
import java.util.HashMap;
import java.util.LinkedList;
import java.util.Scanner;
public class LFRPage {
    static int pageFault(int pages[], int puraPages, int frameSize) {
        HashMap<Integer, Integer> pageFrequency = new HashMap<>();
        LinkedList<Integer> frames = new LinkedList<>();
        int pageFault = 0;
        for (int i = 0; i < puraPages; i++) {
            int page = pages[i];
            if (!frames.contains(page)) {
                pageFault++;
                if (frames.size() < frameSize) {
                    frames.add(page);
                } else {
                    int lfrPage = -1;
                    int minFreq = Integer.MAX_VALUE;
                    for (int f : frames) {
                        int freq = pageFrequency.getOrDefault(f, 0);
                        if (freq < minFreq) {
                            minFreq = freq;
                            lfrPage = f;
                        }
                    }
                    frames.remove((Integer) lfrPage);
                    frames.add(page);
                }
            }
            pageFrequency.put(page, pageFrequency.getOrDefault(page, 0) + 1);
            for (int f : frames) {
                System.out.print(f + " ");
            }
            System.out.println();
        }
    }
}
```

```

    }    return pageFault; }

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.println("Enter the number of pages:");

    int puraPages = sc.nextInt();

    int pages[] = new int[puraPages];

    System.out.println("Enter the pages ");

    for (int p = 0; p < puraPages; p++) {
        pages[p] = sc.nextInt();}

    System.out.println("Enter the number of frames:");

    int frameSize = sc.nextInt();

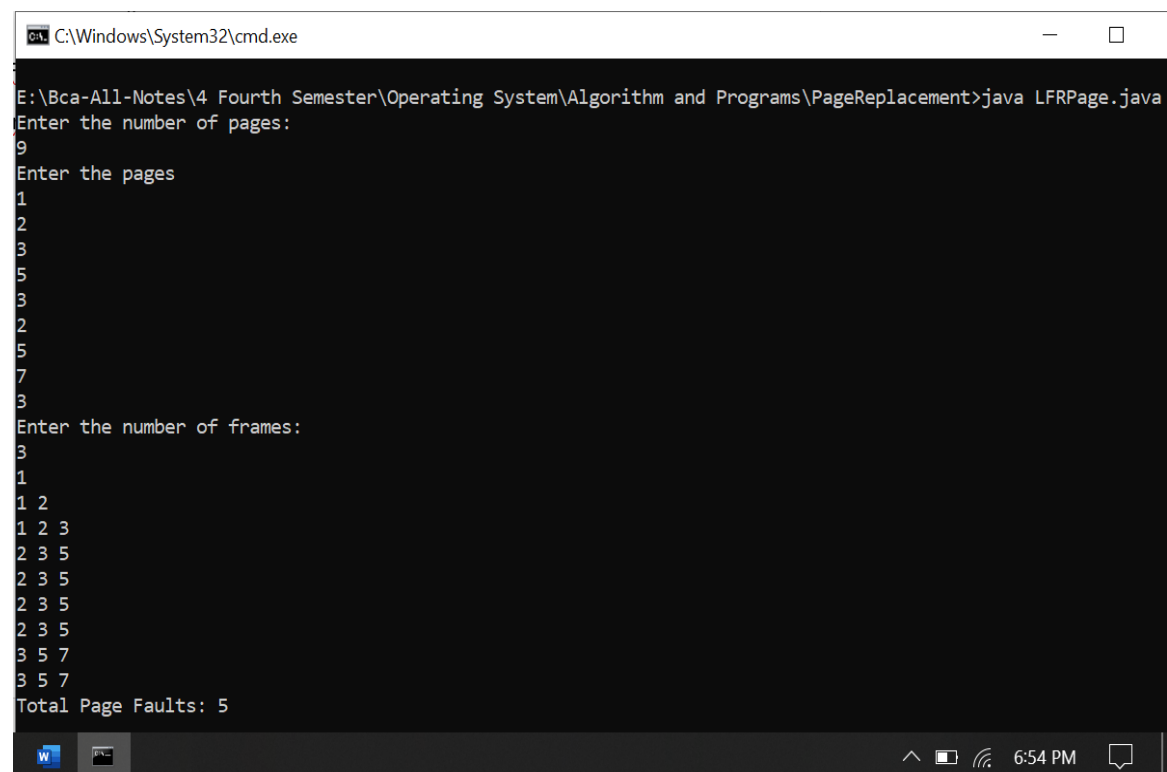
    int totalPageFaults = pageFault(pages, puraPages, frameSize);

    System.out.println("Total Page Faults: " + totalPageFaults);

}
}

```

Output:



```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\PageReplacement>java LFRPage.java
Enter the number of pages:
9
Enter the pages
1
2
3
5
3
2
5
7
3
Enter the number of frames:
3
1
1 2
1 2 3
2 3 5
2 3 5
2 3 5
2 3 5
3 5 7
3 5 7
Total Page Faults: 5

```


6. WAP to implement Clock Page replacement algorithm.

Source Code:

```
import java.util.LinkedList;
import java.util.Scanner;
public class CPR {
    static int pageFault(int pages[], int puraPages, int frameSize) {
        LinkedList<Integer> frames = new LinkedList<>();
        boolean[] refBits = new boolean[frameSize];
        int pageFault = 0, pointer = 0;
        for (int i = 0; i < puraPages; i++) {
            int page = pages[i];
            boolean pageVetiyo = false;
            for (int j = 0; j < frames.size(); j++) {
                if (frames.get(j) == page) {
                    pageVetiyo = true;
                    refBits[j] = true;
                    break;
                }
            }
            if (!pageVetiyo) {
                pageFault++;
                while (true) {
                    if (frames.size() < frameSize) {
                        frames.add(page);
                        refBits[frames.indexOf(page)] = true;
                        break;
                    } else {
                        if (!refBits[pointer] == false) {
                            frames.set(pointer, page);
                            refBits[pointer] = true; break;
                        } else {
                            refBits[pointer] = false;
                        }
                    }
                }
            }
        }
    }
}
```

```

        pointer = (pointer + 1) % frameSize;    } } } }

    for (int f : frames) {
        System.out.print(f + " ");
    }

    System.out.println(); }

    return pageFault; }

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.println("Enter the number of pages:");

    int puraPages = sc.nextInt();

    int pages[] = new int[puraPages];

    System.out.println("Enter the pages ");

    for (int p = 0; p < puraPages; p++) {
        pages[p] = sc.nextInt(); }

    System.out.println("Enter the number of frames:");

    int frameSize = sc.nextInt();

    int totalPageFaults = pageFault(pages, puraPages, frameSize);

    System.out.println("Total Page Faults: " + totalPageFaults);

}

}

```

Output:

```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\PageReplacement>java CPR.java
Enter the number of pages:
7
Enter the pages
1
2
3
5
4
2
1
Enter the number of frames:
3
1
2
3
5
4
2
1
Total Page Faults: 6

```

Disk Scheduling Algorithm

1. WAP to implement First Come First Serve algorithm.

Source Code:

```
import java.util.Scanner;

public class FfcsdiskBg {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the number of requests:");

        int binti = sc.nextInt();

        int requests[] = new int[binti];

        System.out.println("Enter the disk sequence:");

        for (int b = 0; b < binti; b++) {

            requests[b] = sc.nextInt();

            System.out.println("Enter the initial head position:");

            int suruPosition = sc.nextInt();

            int totalHeadMovement = 0;

            int ailekoPosition = suruPosition;

            System.out.println("Disk head movements:");

            for (int m = 0; m < binti; m++) {

                System.out.println("Moving From " + ailekoPosition + " to " + requests[m]);

                totalHeadMovement += Math.abs(ailekoPosition - requests[m]);

                ailekoPosition = requests[m];

            }

            System.out.println("Total head movement is: " + totalHeadMovement);

        }

    }

}
```

Output:

```
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\DiskScheduling>java FfcsdiskBg.java
Enter the number of requests:
5
Enter the disk sequence:
12
34
2
66
90
Enter the initial head position:
2
Disk head movements:
Moving From 2 to 12
Moving From 12 to 34
Moving From 34 to 2
Moving From 2 to 66
Moving From 66 to 90
Total head movement is: 152
```

2. WAP to implement Shortest Seek Time First algorithm.

Source Code:

```
import java.util.Scanner;

public class SSTNdiskBg {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of requests:");
        int binti = sc.nextInt();
        int requests[] = new int[binti];
        System.out.println("Enter the disk sequence:");
        for (int b = 0; b < binti; b++) {
            requests[b] = sc.nextInt();
        }
        System.out.println("Enter the initial head position:");
        int suruPosition = sc.nextInt();
        boolean visited[] = new boolean[binti];
        int totalHeadMovement = 0;
        int ailekoPosition = suruPosition;
        System.out.println("Disk head movements:");
        for (int m = 0; m < binti; m++) {
            int thoriiDistance = Integer.MAX_VALUE;
            int index = -1;
            for (int j = 0; j < binti; j++) {
                if (!visited[j]) {
                    int distance = Math.abs(ailekoPosition - requests[j]);
                    if (distance < thoriiDistance) {
                        thoriiDistance = distance;
                        index = j;
                    }
                }
            }
            // Additional logic for the algorithm would go here
        }
    }
}
```

```

        }
    }
}

visited[index] = true;

System.out.println("Moving From " + ailekoPosition + " to " + requests[index]);

totalHeadMovement += thoriDistance;

ailekoPosition = requests[index];
}

System.out.println("Total head movement is: " + totalHeadMovement);
}
}

```

Output:

```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\DiskScheduling>java SSTNdiskBG.java
Enter the number of requests:
10
Enter the disk sequence:
3
66
2
77
9
78
5
75
44
33
Enter the initial head position:
4
Disk head movements:
Moving From 4 to 3
Moving From 3 to 2
Moving From 2 to 5
Moving From 5 to 9
Moving From 9 to 33
Moving From 33 to 44
Moving From 44 to 66
Moving From 66 to 75
Moving From 75 to 77
Moving From 77 to 78
Total head movement is: 78

```

3. WAP to implement Scan Disk Scheduling Algorithm.

Source Code:

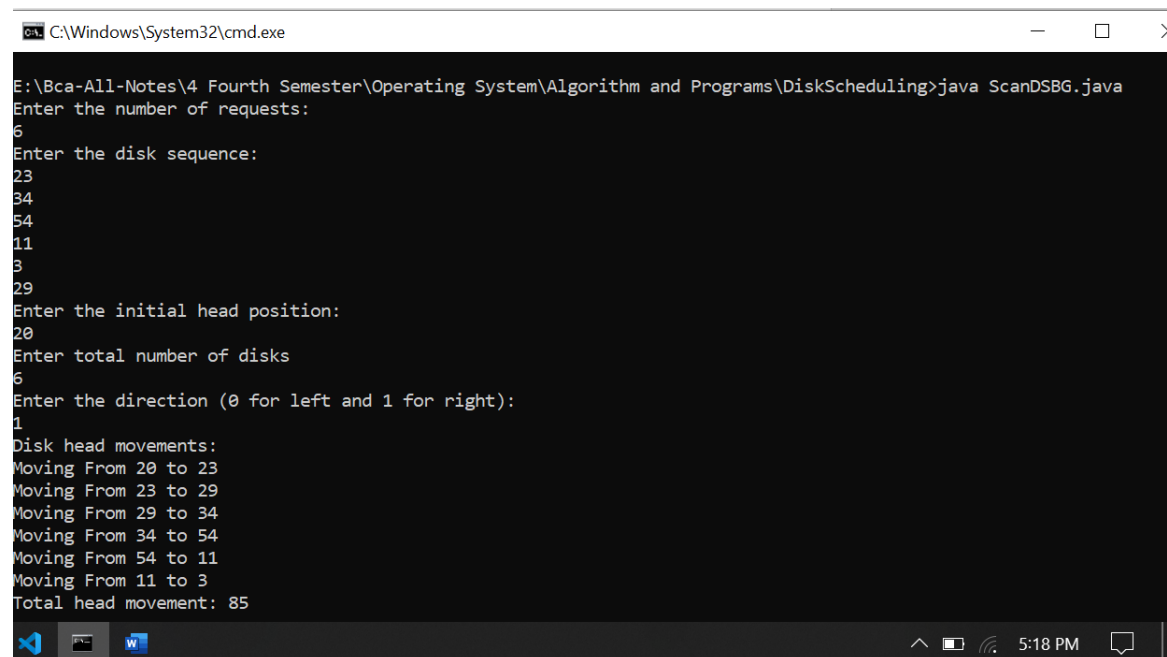
```
import java.util.Arrays;
import java.util.Scanner;
public class ScanDSBG {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of requests:");
        int binti = sc.nextInt();
        int requests[] = new int[binti];
        System.out.println("Enter the disk sequence:");
        for (int b = 0; b < binti; b++) {requests[b] = sc.nextInt();}
        System.out.println("Enter the initial head position:");
        int suruPosition = sc.nextInt();
        System.out.println("Enter total number of disks");
        int totalDisks = sc.nextInt();
        System.out.println("Enter the direction (0 for left and 1 for right):");
        int direction = sc.nextInt();
        Arrays.sort(requests);
        int totalHeadMovement = 0;  int ailekoPosition = suruPosition;
        System.out.println("Disk head movements:");
        if (direction == 1) { for (int j = 0; j < binti; j++) {
            if (requests[j] >= suruPosition) {
                System.out.println("Moving From " + ailekoPosition + " to " + requests[j]);
                totalHeadMovement += Math.abs(ailekoPosition - requests[j]);
                ailekoPosition = requests[j];  } }
            if (ailekoPosition < totalDisks - 1) {
                System.out.println("Moving From " + ailekoPosition + " to " + (totalDisks - 1));
                totalHeadMovement += Math.abs(ailekoPosition - (totalDisks - 1));
                ailekoPosition = totalDisks - 1;}
            for (int k = binti - 1; k >= 0; k--) { if (requests[k] < suruPosition) {
```

```

        System.out.println("Moving From " + ailekoPosition + " to " + requests[k]);
        totalHeadMovement += Math.abs(ailekoPosition - requests[k]);
        ailekoPosition = requests[k]; } } } else {
for (int j = binti - 1; j >= 0; j--) { if (requests[j] <= suruPosition) {
    System.out.println("Moving From " + ailekoPosition + " to " + requests[j]);
    totalHeadMovement += Math.abs(ailekoPosition - requests[j]);
    ailekoPosition = requests[j]; } } if (ailekoPosition > 0) {
System.out.println("Moving From " + ailekoPosition + " to " + 0);
totalHeadMovement += Math.abs(ailekoPosition - 0);
ailekoPosition = 0; }
for (int k = 0; k < binti; k++) { if (requests[k] > suruPosition) {
    System.out.println("Moving From " + ailekoPosition + " to " + requests[k]);
    totalHeadMovement += Math.abs(ailekoPosition - requests[k]);
    ailekoPosition = requests[k]; } } }
System.out.println("Total head movement: " + totalHeadMovement);
sc.close(); }
}

```

Output:



```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\DiskScheduling>java ScanDSBG.java
Enter the number of requests:
6
Enter the disk sequence:
23
34
54
11
3
29
Enter the initial head position:
20
Enter total number of disks
6
Enter the direction (0 for left and 1 for right):
1
Disk head movements:
Moving From 20 to 23
Moving From 23 to 29
Moving From 29 to 34
Moving From 34 to 54
Moving From 54 to 11
Moving From 11 to 3
Total head movement: 85

```

4. WAP to implement C-Scan Disk Scheduling Algorithm.

Source Code:

```
import java.util.Arrays;
import java.util.Scanner;
public class CScanBg {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of requests:");
        int binti = sc.nextInt();
        int requests[] = new int[binti];
        System.out.println("Enter the disk sequence:");
        for (int b = 0; b < binti; b++) {
            requests[b] = sc.nextInt();
        }
        System.out.println("Enter the initial head position:");
        int suruPosition = sc.nextInt();
        System.out.println("Enter total number of disks");
        int totalDisks = sc.nextInt();
        Arrays.sort(requests);
        int totalHeadMovement = 0;
        int ailekoPosition = suruPosition;
        System.out.println("Disk head movements:");
        for (int j = 0; j < binti; j++) {
            if (requests[j] >= suruPosition) {
                System.out.println("Moving From " + ailekoPosition + " to " + requests[j]);
                totalHeadMovement += Math.abs(ailekoPosition - requests[j]);
                ailekoPosition = requests[j];
            }
        }
        if (ailekoPosition < totalDisks - 1) {
            System.out.println("Moving From " + ailekoPosition + " to " + (totalDisks - 1));
```



```

        totalHeadMovement += Math.abs(ailekoPosition - (totalDisks - 1));

        ailekoPosition = totalDisks - 1;
    }

    System.out.println("Moving From " + (totalDisks - 1) + " to 0");

    totalHeadMovement += (totalDisks - 1);

    ailekoPosition = 0;

    for (int k = 0; k < binti; k++) {

        if (requests[k] < suruPosition) {

            System.out.println("Moving From " + ailekoPosition + " to " + requests[k]);

            totalHeadMovement += Math.abs(ailekoPosition - requests[k]);

            ailekoPosition = requests[k];

        }

    }

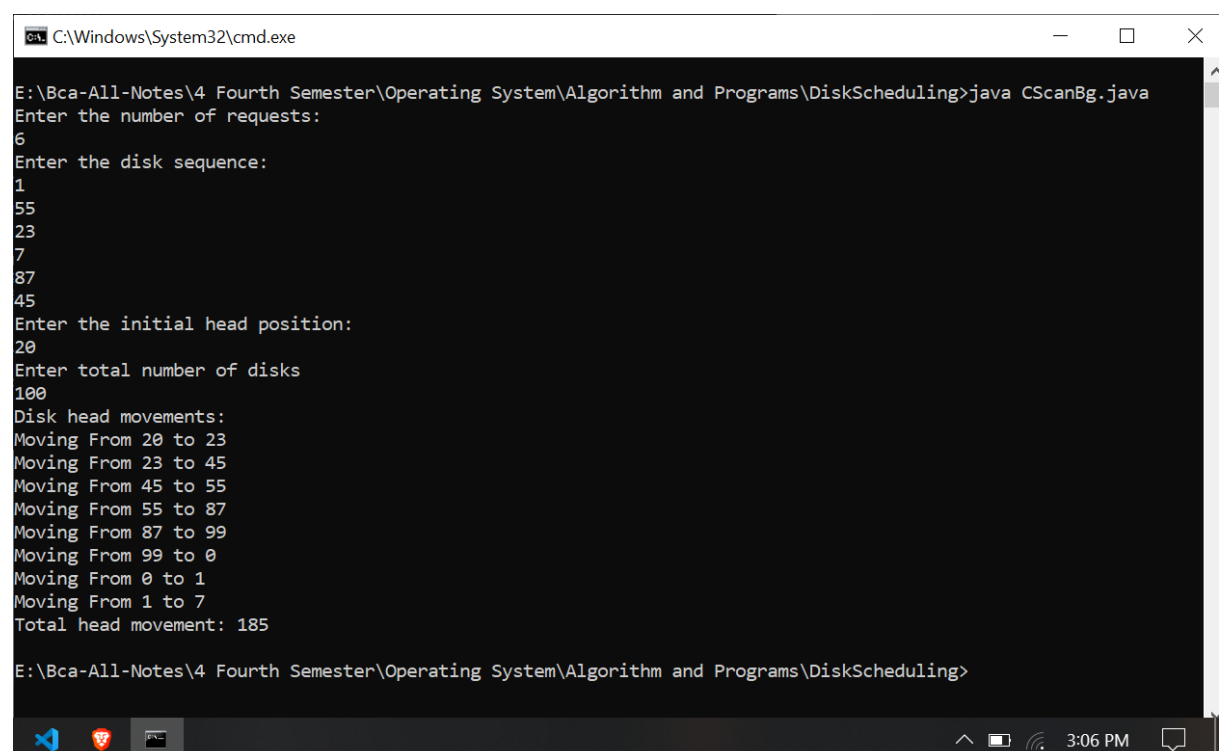
    System.out.println("Total head movement: " + totalHeadMovement);

}

}

```

Output:



```

C:\Windows\System32\cmd.exe

E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\DiskScheduling>java CScanBg.java
Enter the number of requests:
6
Enter the disk sequence:
1
55
23
7
87
45
Enter the initial head position:
20
Enter total number of disks
100
Disk head movements:
Moving From 20 to 23
Moving From 23 to 45
Moving From 45 to 55
Moving From 55 to 87
Moving From 87 to 99
Moving From 99 to 0
Moving From 0 to 1
Moving From 1 to 7
Total head movement: 185

E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\DiskScheduling>

```

5. WAP to implement Look Disk Scheduling Algorithm.

Source code:

```
import java.util.Arrays;
import java.util.Scanner;
public class LookDSBG {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of requests:");
        int binti = sc.nextInt();
        int requests[] = new int[binti];
        System.out.println("Enter the disk sequence:");
        for (int b = 0; b < binti; b++) {
            requests[b] = sc.nextInt();
        }
        System.out.println("Enter the initial head position:");
        int suruPosition = sc.nextInt();
        System.out.println("Enter the direction (0 for left and 1 for right):");
        int direction = sc.nextInt();
        Arrays.sort(requests);
        int totalHeadMovement = 0;
        int ailekoPosition = suruPosition;
        System.out.println("Disk head movements:");
        if (direction == 1) {
            for (int j = 0; j < binti; j++) {
                if (requests[j] >= suruPosition) {
                    System.out.println("Moving From " + ailekoPosition + " to " + requests[j]);
                    totalHeadMovement += Math.abs(ailekoPosition - requests[j]);
                    ailekoPosition = requests[j];
                }
            }
            for (int k = binti - 1; k >= 0; k--) {
                if (requests[k] < suruPosition) {
```

```

        System.out.println("Moving From " + ailekoPosition + " to " + requests[k]);
        totalHeadMovement += Math.abs(ailekoPosition - requests[k]);
        ailekoPosition = requests[k]; }
    }}
else {
    for (int j = binti - 1; j >= 0; j--) {
        if (requests[j] <= suruPosition) {
            System.out.println("Moving From " + ailekoPosition + " to " + requests[j]);
            totalHeadMovement += Math.abs(ailekoPosition - requests[j]);
            ailekoPosition = requests[j];}
    }

    for (int k = 0; k < binti; k++) {
        if (requests[k] > suruPosition) {
            System.out.println("Moving From " + ailekoPosition + " to " + requests[k]);
            totalHeadMovement += Math.abs(ailekoPosition - requests[k]);
            ailekoPosition = requests[k]; } } }

    System.out.println("Total head movement: " + totalHeadMovement);
    sc.close();
}
}

```

Output:

```

Select C:\Windows\System32\cmd.exe

E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\DiskScheduling>java LookDSBG.java
Enter the number of requests:
5
Enter the disk sequence:
22
1
90
67
87
Enter the initial head position:
44
Enter the direction (0 for left and 1 for right):
1
Disk head movements:
Moving From 44 to 67
Moving From 67 to 87
Moving From 87 to 90
Moving From 90 to 22
Moving From 22 to 1
Total head movement: 135

```

6. WAP to implement C-Look disk scheduling algorithm.

Source Code:

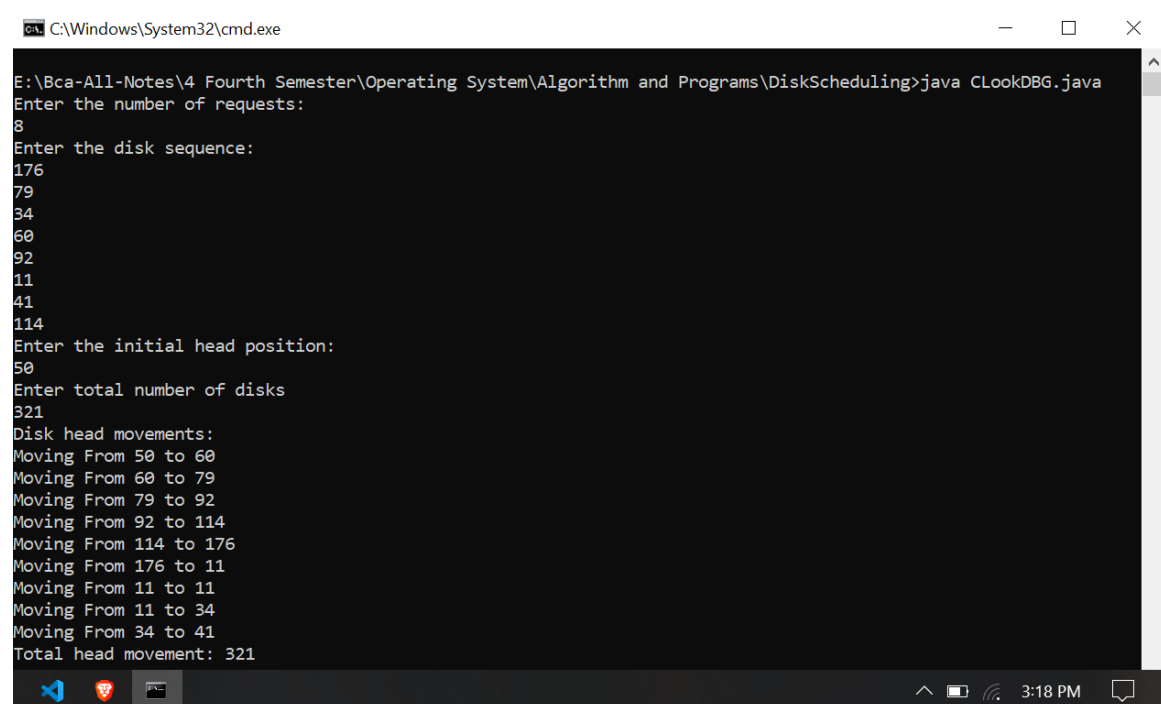
```
import java.util.Arrays;
import java.util.Scanner;
public class CLookDBG {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of requests:");
        int binti = sc.nextInt();
        int requests[] = new int[binti];
        System.out.println("Enter the disk sequence:");
        for (int b = 0; b < binti; b++) {
            requests[b] = sc.nextInt();
        }
        System.out.println("Enter the initial head position:");
        int suruPosition = sc.nextInt();
        System.out.println("Enter total number of disks");
        int totalDisks = sc.nextInt();
        Arrays.sort(requests);
        int totalHeadMovement = 0;
        int ailekoPosition = suruPosition;
        System.out.println("Disk head movements:");
        for (int j = 0; j < binti; j++) {
            if (requests[j] >= suruPosition) {
                System.out.println("Moving From " + ailekoPosition + " to " + requests[j]);
                totalHeadMovement += Math.abs(ailekoPosition - requests[j]);
                ailekoPosition = requests[j];
            }
        }
    }
}
```

```

    if (ailekoPosition > requests[0]) {
        System.out.println("Moving From " + ailekoPosition + " to " + requests[0]);
        totalHeadMovement += Math.abs(ailekoPosition - requests[0]);
        ailekoPosition = requests[0];
    }
    for (int k = 0; k < binti; k++) {
        if (requests[k] < suruPosition) {
            System.out.println("Moving From " + ailekoPosition + " to " + requests[k]);
            totalHeadMovement += Math.abs(ailekoPosition - requests[k]);
            ailekoPosition = requests[k];
        }
    }
    System.out.println("Total head movement: " + totalHeadMovement);
}
}

```

Output:



```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\DiskScheduling>java CLookDBG.java
Enter the number of requests:
8
Enter the disk sequence:
176
79
34
60
92
11
41
114
Enter the initial head position:
50
Enter total number of disks
321
Disk head movements:
Moving From 50 to 60
Moving From 60 to 79
Moving From 79 to 92
Moving From 92 to 114
Moving From 114 to 176
Moving From 176 to 11
Moving From 11 to 11
Moving From 11 to 34
Moving From 34 to 41
Total head movement: 321

```

Memory Allocation Algorithm

1. WAP to implement First Fit Memory Allocation Algorithm.

Source Code:

```
import java.util.Scanner;

public class PailoFit {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the number of memory slot");

        int slots = sc.nextInt();

        int[] memorySizes = new int[slots];

        boolean[] filled = new boolean[slots];

        for (int b = 0; b < slots; b++) {

            System.out.println("Enter the size of memory slot " + (b + 1));

            memorySizes[b] = sc.nextInt();

        }

        System.out.println("Enter the number of processes");

        int processes = sc.nextInt();

        int[] processSizes = new int[processes];

        int allocation[] = new int[processes];

        for (int b = 0; b < processes; b++) {

            System.out.println("Enter the size of process " + (b + 1));

            processSizes[b] = sc.nextInt();

            allocation[b] = -1;

        }

        for (int i = 0; i < processes; i++) {

            for (int j = 0; j < slots; j++) {

                if (!filled[j] && memorySizes[j] >= processSizes[i]) {

                    allocation[i] = j;

                    filled[j] = true;

                    break;

                }

            }

        }

    }

}
```

```

        }
    }
}

System.out.println("Process No.\tProcess Size\tMemory Slot No.");
for (int i = 0; i < processes; i++) {
    System.out.print((i + 1) + "\t\t" + processSizes[i] + "\t\t");
    if (allocation[i] != -1) {
        System.out.println((allocation[i] + 1));
    } else {
        System.out.println("Not Allocated");
    }
}
}
sc.close();
}
}

```

Output:

```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\Memory Allocation>java Pailofit.java
Enter the number of memory slot
3
Enter the size of memory slot 1
400
Enter the size of memory slot 2
100
Enter the size of memory slot 3
200
Enter the number of processes
4
Enter the size of process 1
300
Enter the size of process 2
99
Enter the size of process 3
232
Enter the size of process 4
120
Process No.    Process Size    Memory Slot No.
1              300            1
2              99             2
3              232            Not Allocated
4              120            3
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\Memory Allocation>

```

2. WAP to implement Best Fit Memory Allocation Algorithm.

Source Code:

```
import java.util.Scanner;

public class SaiFitBG {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the number of memory slot");

        int slots = sc.nextInt();

        int []memorySizes= new int [slots];

        boolean[]filled = new boolean[slots];

        for (int b=0;b<slots;b++){

            System.out.println("Enter the size of memory slot "+(b+1));

            memorySizes[b]=sc.nextInt();

        }

        System.out.println("Enter the number of processes");

        int processes = sc.nextInt();

        int []processSizes= new int [processes];

        int []allocation = new int[processes];

        for (int b=0;b<processes;b++){

            System.out.println("Enter the size of process "+(b+1));

            processSizes[b]=sc.nextInt();

            allocation[b]=-1;

        }

        for (int i=0;i<processes;i++){

            int bestIdx=-1;

            for (int j=0;j<slots;j++){

                if (!filled[j] && memorySizes[j]>=processSizes[i]){

                    if (bestIdx==-1){

                        bestIdx=j;

                    }

                    else if (memorySizes[j]<memorySizes[bestIdx]){


```



```

        bestIdx=j;
    }} }

    if (bestIdx!=-1){
        allocation[i]=bestIdx;
memorySizes[bestIdx] -= processSizes[i];}

    }

    System.out.println("Process No.\tProcess Size\tMemory Slot No.");
    for (int i=0;i<processes;i++){
        System.out.print((i+1)+"\t\t"+processSizes[i]+" \t\t");
        if (allocation[i]!=-1){
            System.out.println((allocation[i]+1)); }
        else {
            System.out.println("Not Allocated");
        } }
    sc.close();
}
}

```

Output:

```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\Memory Allocation>java SaiFitBG.java
Enter the number of memory slot
3
Enter the size of memory slot 1
100
Enter the size of memory slot 2
500
Enter the size of memory slot 3
300
Enter the number of processes
6
Enter the size of process 1
50
Enter the size of process 2
30
Enter the size of process 3
150
Enter the size of process 4
200
Enter the size of process 5
300
Enter the size of process 6
20
Process No.    Process Size    Memory Slot No.
1             50             1
2             30             1
3            150             3
4            200             2
5            300             2
6             20             1

```

3. WAP to implement Worst Fit Memory Allocation Algorithm.

Source code:

```
import java.util.Scanner;

public class WorstFitBG{

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the number of memory slot");

        int slots = sc.nextInt();

        int []memorySizes= new int [slots];

        boolean[]filled = new boolean[slots];

        for (int b=0;b<slots;b++){

            System.out.println("Enter the size of memory slot "+(b+1));

            memorySizes[b]=sc.nextInt();

        }

        System.out.println("Enter the number of processes");

        int processes = sc.nextInt();

        int []processSizes= new int [processes];

        int []allocation = new int[processes];

        for (int b=0;b<processes;b++){

            System.out.println("Enter the size of process "+(b+1));

            processSizes[b]=sc.nextInt();

            allocation[b]=-1;    }

        for (int i = 0; i < processes; i++) {

            int worstIdx = -1;

            for (int j = 0; j < slots; j++) {

                if (!filled[j] && memorySizes[j] >= processSizes[i]) {

                    if (worstIdx == -1 || memorySizes[j] > memorySizes[worstIdx]) {

                        worstIdx = j;

                    }

                }

            }

        }

    }

}
```

```

    }

    if (worstIdx != -1) {
        allocation[i] = worstIdx;
        memorySizes[worstIdx] -= processSizes[i];
    }
}

System.out.println("Process No.\tProcess Size\tMemory Slot No.");
for (int i = 0; i < processes; i++) {
    System.out.print((i + 1) + "\t\t" + processSizes[i] + "\t\t");
    if (allocation[i] != -1) {
        System.out.println((allocation[i] + 1));
    } else {
        System.out.println("Not Allocated");
    }
}

sc.close();
}
}

```

Output:

```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\Memory Allocation>java WorstFitBG.java
Enter the number of memory slot
2
Enter the size of memory slot 1
500
Enter the size of memory slot 2
200
Enter the number of processes
4
Enter the size of process 1
100
Enter the size of process 2
300
Enter the size of process 3
200
Enter the size of process 4
5
Process No.    Process Size    Memory Slot No.
1             100            1
2             300            1
3             200            2
4              5             1
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\Memory Allocation>

```

Deadlocks

1. WAP to implement Bankers Algorithm.

Source code:

```
import java.util.Scanner;

public class BankersBG {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the number of processes");
        int processNum = sc.nextInt();
        System.out.println("Enter the number of resources");
        int resourceNum = sc.nextInt();
        int[][] max = new int[processNum][resourceNum];
        int[][] allocation = new int[processNum][resourceNum];
        int[][] chiyeko = new int[processNum][resourceNum];
        int[] vako = new int[resourceNum];
        boolean[] visited = new boolean[processNum];
        int[] safe = new int[processNum];
        System.out.println("Enter the max matrix");
        for (int i = 0; i < processNum; i++) {
            for (int j = 0; j < resourceNum; j++) {
                max[i][j] = sc.nextInt();
            }
        }
        System.out.println("Enter the allocation matrix");
        for (int i = 0; i < processNum; i++) {
            for (int j = 0; j < resourceNum; j++) {
                allocation[i][j] = sc.nextInt();
                chiyeko[i][j] = max[i][j] - allocation[i][j];
            }
        }
    }
}
```

```

    }

    System.out.println("Enter the available resources");

    for (int i = 0; i < resourceNum; i++) {
        vako[i] = sc.nextInt();
    }

    int c = 0;

    System.out.println("Safe sequence is:");

    while (c < processNum) {
        boolean vitiyo = false;

        for (int i = 0; i < processNum; i++) {
            if (visited[i] == false) {
                int j;

                for (j = 0; j < resourceNum; j++) {
                    if (chiyeke[i][j] > vako[j]) {

                        break;
                    }
                }

                if (j == resourceNum) {
                    for (int k = 0; k < resourceNum; k++) {
                        vako[k] += allocation[i][k];
                    }

                    safe[c++] = i;
                    visited[i] = true;
                    vitiyo = true;
                }
            }
        }

        if (!vitiyo) {

            System.out.println("The system is not in safe state");
        }
    }
}

```

```

        return;
    }
}

System.out.print("Hence, the SAFE Sequence is as follows: ");
for (int i = 0; i < processNum; i++) {
    System.out.print("P" + safe[i]);
    if (i != processNum - 1) {
        System.out.print(" -> ");
    }
}

System.out.println(".");
sc.close();
}
}

```

Output:

```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\DeadLock>java BankersBG.java
Enter the number of processes
3
Enter the number of resources
3
Enter the max matrix
7
5
3
3
2
2
9
0
2
Enter the allocation matrix
0
1
0
2
0
0
3
0
2
Enter the available resources
3
3
2
Safe sequence is:
The system is not in safe state

```

2. WAP to implement deadlock detection algorithm.

Code:

```
import java.util.Scanner;

public class DeadLockDetectionBg {
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the number of processes");
        int processNum = sc.nextInt();

        System.out.println("Enter the number of resources");
        int resourceNum = sc.nextInt();

        int[][] allocation = new int[processNum][resourceNum];
        int[][] chiyeko = new int[processNum][resourceNum];
        int[] vako = new int[resourceNum];
        boolean[] visited = new boolean[processNum];

        System.out.println("Enter the allocation matrix");
        for (int i = 0; i < processNum; i++) {
            for (int j = 0; j < resourceNum; j++) {
                allocation[i][j] = sc.nextInt();
            }
        }

        System.out.println("Enter the request resources");
        for (int i = 0; i < resourceNum; i++) {
            for (int j = 0; j < processNum; j++) {
                chiyeko[i][j] = sc.nextInt();
            }
        }
    }
}
```

```

    }

    System.out.println("Enter the available resources");

    for (int i = 0; i < resourceNum; i++) {
        vako[i] = sc.nextInt();
    }

    if (isDeadLock(allocation, chiyeko, vako, visited, processNum, resourceNum)) {
        System.out.println("The system is in deadlock state");
    } else {
        System.out.println("The system is not in deadlock state");
    }
}

public static boolean isDeadLock(int[][] allocation, int[][] chiyeko, int[] vako, boolean[]
visited,
    int processNum, int resourceNum) {
    int c = 0;
    while (c < processNum) {
        boolean vitiyo = false;
        for (int i = 0; i < processNum; i++) {
            if (visited[i] == false) {
                int j;
                for (j = 0; j < resourceNum; j++) {
                    if (chiyeko[i][j] > vako[j]) {

                        break;
                    }
                }
                if (j == resourceNum) {
                    for (int k = 0; k < resourceNum; k++) {
                        vako[k] += allocation[i][k];
                    }
                }
            }
        }
    }
}

```



```

        visited[i] = true;
        vitiyo = true;
        c++;
    }
}
}
if (!vitiyo) {
    return true;
}
}
return false;

}
}

```

Output:

```

C:\Windows\System32\cmd.exe

E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\DeadLock>java DeadLockDetectionBg.java
Enter the number of processes
2
Enter the number of resources
2
Enter the allocation matrix
1
0
0
1
Enter the request resources
0
1
1
0
Enter the available resources
0
0
0
The system is in deadlock state

E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\DeadLock>

```

3. WAP to implement deadlock recovery.

```
import java.util.Scanner;

public class DeadLockRecovery {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the number of processes");
        int processNum = sc.nextInt();

        System.out.println("Enter the number of resources");
        int resourceNum = sc.nextInt();

        int[][] allocation = new int[processNum][resourceNum];
        int[][] chiyeko = new int[processNum][resourceNum];
        int[] vako = new int[resourceNum];
        boolean[] visited = new boolean[processNum];

        System.out.println("Enter the allocation matrix");
        for (int i = 0; i < processNum; i++) {
            for (int j = 0; j < resourceNum; j++) {
                allocation[i][j] = sc.nextInt();
            }
        }

        System.out.println("Enter the request resources");
        for (int i = 0; i < processNum; i++) {
            for (int j = 0; j < resourceNum; j++) {
                chiyeko[i][j] = sc.nextInt();
            }
        }

        System.out.println("Enter the available resources");
        for (int i = 0; i < resourceNum; i++) {
```

```

        vako[i] = sc.nextInt();
    }

    boolean deadlockExists = isDeadlock(allocation, chiyeko, vako, visited,
processNum, resourceNum);

    if (deadlockExists) {
        System.out.println("\nDeadlock Detected!");
        System.out.println("Starting Recovery Process...\n");
        boolean[] visited2 = new boolean[processNum];
        deadlockrecovery(allocation, chiyeko, vako, visited2, processNum,
resourceNum);
    } else {
        System.out.println("\nNo Deadlock.  System is Safe.");
    }
}

public static boolean isDeadlock(int[][] allocation, int[][] chiyeko, int[] vako, boolean[]
visited,
    int processNum, int resourceNum) {
    int c = 0;
    int[] vakoTemp = new int[resourceNum];
    for (int i = 0; i < resourceNum; i++) {
        vakoTemp[i] = vako[i];
    }

    while (c < processNum) {
        boolean vitiyo = false;
        for (int i = 0; i < processNum; i++) {
            if (visited[i] == false) {
                int j;

```

```

        for (j = 0; j < resourceNum; j++) {
            if (chiyeke[i][j] > vakoTemp[j]) {
                break;
            }
        }
        if (j == resourceNum) {
            for (int k = 0; k < resourceNum; k++) {
                vakoTemp[k] += allocation[i][k];
            }
            visited[i] = true;
            vitiyo = true;
            c++;
        }
    }
    if (!vitiyo) {
        return true;
    }
    return false;
}

```

```

public static void deadlockrecovery(int[][] allocation, int[][] chiyeke, int[] vako,
boolean[] visited,

```

```

    int processNum, int resourceNum) {
    int c = 0;
    int terminatedCount = 0;

```

```

    while (c < processNum) {
        boolean vitiyo = false;
        for (int i = 0; i < processNum; i++) {

```

```

    if (visited[i] == false) {
        int j;
        for (j = 0; j < resourceNum; j++) {
            if (chiyekeo[i][j] > vako[j]) {
                break;
            }
        }
        if (j == resourceNum) {
            for (int k = 0; k < resourceNum; k++) {
                vako[k] += allocation[i][k];
            }
            visited[i] = true;
            vitiyo = true;
            c++;
            terminatedCount++;
            System.out.println("Process " + i + " is terminated to recover from
deadlock.");
        }
    }
}

if (!vitiyo) {
    for (int i = 0; i < processNum; i++) {
        if (visited[i] == false) {
            System.out.println("Process " + i + " is terminated to recover from
deadlock.");
            for (int k = 0; k < resourceNum; k++) {
                vako[k] += allocation[i][k];
            }
            visited[i] = true;
            terminatedCount++;
            c++;

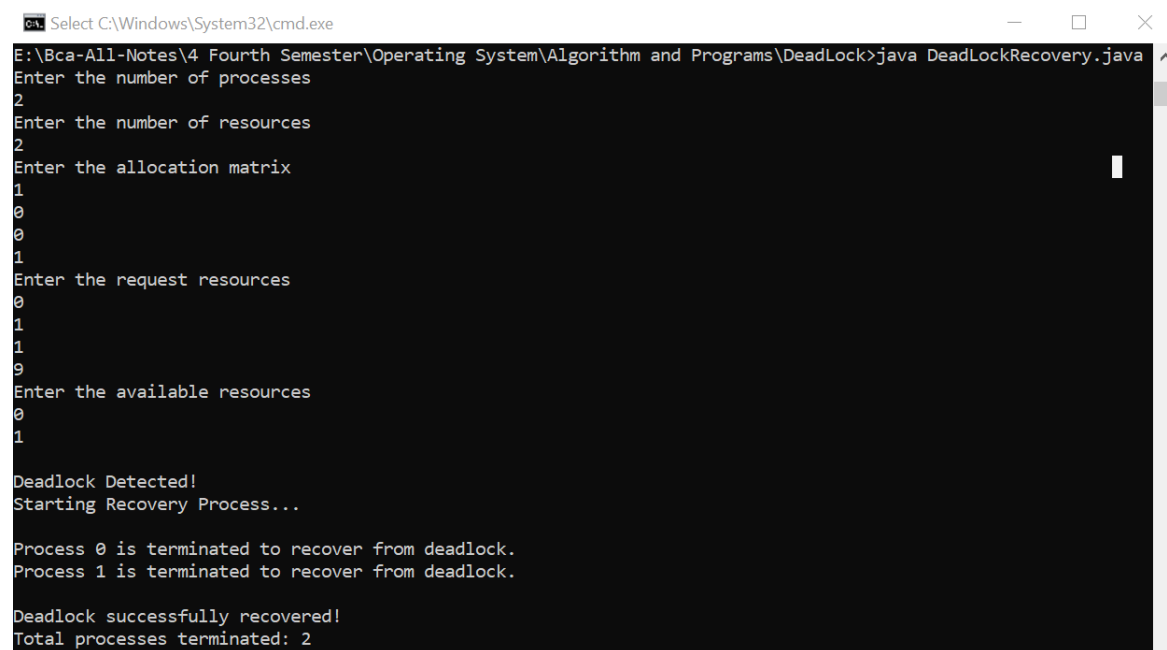
```

```

        }
    }
    if (c < processNum) {
        System.out.println("\nDeadlock cannot be resolved further.");
    } else {
        System.out.println("\nDeadlock successfully recovered!");
        System.out.println("Total processes terminated: " + terminatedCount);
    }
    return;
}
}

System.out.println("\nDeadlock successfully recovered!");
System.out.println("Total processes terminated: " + terminatedCount);
}
}

```



The screenshot shows a Windows command prompt window titled "Select C:\Windows\System32\cmd.exe". The window displays the execution of a Java program named "DeadLockRecovery.java". The program prompts the user to enter the number of processes (2), the number of resources (2), the allocation matrix (1 0 0 1), the request resources (0 1 1 9), and the available resources (0 1). The program then detects a deadlock and starts a recovery process. It terminates process 0 and process 1 to recover from the deadlock. Finally, it reports that the deadlock was successfully recovered and that a total of 2 processes were terminated.

```

E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\DeadLock>java DeadLockRecovery.java
Enter the number of processes
2
Enter the number of resources
2
Enter the allocation matrix
1
0
0
1
Enter the request resources
0
1
1
9
Enter the available resources
0
1

Deadlock Detected!
Starting Recovery Process...

Process 0 is terminated to recover from deadlock.
Process 1 is terminated to recover from deadlock.

Deadlock successfully recovered!
Total processes terminated: 2

```

Classical IPC Problems

1. WAP to implement Dining Philosopher Problem Algorithm.

Code:

```
import java.util.Scanner;

public class DiningBg {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the number of Philosophers:");

        int n = sc.nextInt();

        Philosopher[] philosophers = new Philosopher[n];

        Fork[] forks = new Fork[n];

        for (int i = 0; i < n; i++) {

            forks[i] = new Fork(i);

        }

        for (int i = 0; i < n; i++) {

            philosophers[i] = new Philosopher(i, forks[i], forks[(i + 1) % n]);

            new Thread(philosophers[i]).start();

        }

        System.out.println("Philosophers are dining...");

        sc.close();

    }

}

class Fork {

    public final int id;

    private boolean prayog;

    public Fork(int id) {

        this.id = id;

        this.prayog = false;

    }

    public synchronized void pickUp(int philosopherId) throws InterruptedException {

        while (prayog) {

        }

    }

}
```

```

        wait();
    }

    prayog = true;

    System.out.println("Philosopher " + philosopherId + " picked up Fork " + id);
}

public synchronized void putDown() {
    prayog = false;

    System.out.println("Fork " + id + " is put down.");

    notifyAll();
}
}

class Philosopher implements Runnable {
    private final int id;
    private final Fork dabareFork;
    private final Fork rightFork;

    public Philosopher(int id, Fork dabareFork, Fork rightFork) {
        this.id = id;
        this.dabareFork = dabareFork;
        this.rightFork = rightFork;
    }

    @Override
    public void run() {
        try {
            while (true) {
                think();
                eat();
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

```



```

    }

    private void think() throws InterruptedException {
        System.out.println("Philosopher " + id + " is thinking.");
        Thread.sleep((long) (Math.random() * 1000));
    }

    private void eat() throws InterruptedException {
        Fork first = dabareFork.id < rightFork.id ? dabareFork : rightFork;
        Fork second = dabareFork.id < rightFork.id ? rightFork : dabareFork;
        first.pickUp(id);
        second.pickUp(id);
        System.out.println("Philosopher " + id + " is eating.");
        Thread.sleep((long) (Math.random() * 1000));
        first.putDown();
        second.putDown();
    }
}

```

```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\Classical IPC Problem>javac DiningBg.java
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\Classical IPC Problem>java DiningBg.java
Enter the number of Philosophers:
2
Philosophers are dining...
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 0 picked up Fork 0
Philosopher 0 picked up Fork 1
Philosopher 0 is eating.
Fork 0 is put down.
Fork 1 is put down.
Philosopher 0 is thinking.
Philosopher 1 picked up Fork 0
Philosopher 1 picked up Fork 1
Philosopher 1 is eating.
Fork 0 is put down.
Fork 1 is put down.
Philosopher 1 is thinking.
Philosopher 0 picked up Fork 0
Philosopher 0 picked up Fork 1
Philosopher 0 is eating.
Fork 0 is put down.
Fork 1 is put down.
Philosopher 0 is thinking.
Philosopher 1 picked up Fork 0
Philosopher 1 picked up Fork 1
Philosopher 1 is eating.
Fork 0 is put down.
Fork 1 is put down.
Philosopher 1 is thinking.

```

2. WAP to implement Readers and Writes Problem Algorithm.

Code:

```
import java.util.Scanner;

public class PadauniAndLekhauniSamasShya {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the number of Padauni Manxe");

        int padauni = sc.nextInt();

        System.out.println("Enter the number of Lekhauni Manxe");

        int lekhauni = sc.nextInt();

        BadekoData badekoData = new BadekoData(padauni, lekhauni);

        for (int i = 0; i < padauni; i++) {

            int padauniId = i;

            new Thread() {public void run() {

                try {

                    badekoData.padauniKaam(padauniId);

                } catch (InterruptedException e) {

                    e.printStackTrace();

                } }.start();

            }

        }

        for (int i = 0; i < lekhauni; i++) {

            int lekhauniId = i;

            new Thread() {

                public void run() {

                    try {

                        badekoData.lekhauniKaam(lekhauniId);

                    } catch (InterruptedException e) {

                        e.printStackTrace();

                    }

                }

            }.start();

        }

        sc.close();

    }

}

class BadekoData {

    private int padauni;

    private int lekhauni;

    private int padauniCount = 0;

    private boolean lekhauniActive = false;

    public BadekoData(int padauni, int lekhauni) {

        this.padauni = padauni;

    }

}
```

```

        this.lekhauni = lekhauni;
    }

    public synchronized void padauniKaam(int id) throws InterruptedException {
        while (lekhauniActive) {
            wait(); }

        padauniCount++;

        System.out.println("Padauni Manxe " + id + " is reading. Active readers: " +
padauniCount);

        Thread.sleep((long) (Math.random() * 1000));

        padauniCount--;

        System.out.println("Padauni Manxe " + id + " finished reading. Active readers: " +
padauniCount);

        if (padauniCount == 0) {
            notifyAll();} }

    public synchronized void lekhauniKaam(int id) throws InterruptedException {
        while (lekhauniActive || padauniCount > 0) { wait();}

        lekhauniActive = true;

        System.out.println("Lekhauni Manxe " + id + " is writing.");

        Thread.sleep((long) (Math.random() * 1000));

        lekhauniActive = false;

        System.out.println("Lekhauni Manxe " + id + " finished writing.");

        notifyAll(); }
}

```

```

C:\Windows\System32\cmd.exe
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\Classical IPC Problem>javac PadauniAndLekhauniSamasShya.java
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\Classical IPC Problem>java PadauniAndLekhauniSamasShya.java
Enter the number of Padauni Manxe
3
Enter the number of Lekhauni Manxe
2
Padauni Manxe 0 is reading. Active readers: 1
Padauni Manxe 0 finished reading. Active readers: 0
Lekhauni Manxe 0 is writing.
Lekhauni Manxe 0 finished writing.
Padauni Manxe 2 is reading. Active readers: 1
Padauni Manxe 2 finished reading. Active readers: 0
Lekhauni Manxe 1 is writing.
Lekhauni Manxe 1 finished writing.
Padauni Manxe 1 is reading. Active readers: 1
Padauni Manxe 1 finished reading. Active readers: 0
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\Classical IPC Problem>

```

3. WAP to implement Sleep Barbers problem algorithm.

Code:

```
import java.util.Scanner;

import java.util.concurrent.Semaphore;

public class SleepingBarberBg {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter the number of chairs in waiting room:");

        int waitingChairs = sc.nextInt();

        System.out.println("Enter the number of Customers:");

        int customerCount = sc.nextInt();

        BarberShop barberShop = new BarberShop(waitingChairs);

        new Thread(() -> barberShop.barberKam()).start();

        for (int i = 0; i < customerCount; i++) {    int customerId = i;

            new Thread(() -> barberShop.customerArrivers(customerId)).start();

            try {                Thread.sleep((long) (Math.random() * 500));

                } catch (InterruptedException e) {                e.printStackTrace();

                }} }}

class BarberShop {

    private final Semaphore barberReady;

    private final Semaphore customerReady;

    private final Semaphore accessSeats;

    private int khaliChairs;

    public BarberShop(int khurchi) {

        this.barberReady = new Semaphore(0);

        this.customerReady = new Semaphore(0);

        this.accessSeats = new Semaphore(1);

        this.khaliChairs = khurchi;    }

    public void barberKam() {

        while (true) {    try {

            customerReady.acquire();
```

```

        accessSeats.acquire(); khaliChairs++;

        barberReady.release(); accessSeats.release();

        System.out.println("Barber is cutting hair.");

        Thread.sleep((long) (Math.random() * 1000));

    } catch (InterruptedException e) {

        e.printStackTrace(); } } }

public void customerArrivers(int id) {

    try { accessSeats.acquire();

        if(khaliChairs > 0) { khaliChairs--;

            System.out.println("Customer " + id + " is waiting. Available chairs: " +
khaliChairs);

            customerReady.release(); accessSeats.release(); barberReady.acquire();

            System.out.println("Customer " + id + " is getting a haircut.");

            Thread.sleep((long) (Math.random() * 1000));

            System.out.println("Customer " + id + " is done with the haircut.");

        } else { accessSeats.release();

            System.out.println("Customer " + id + " leaves as no chairs are
available."); }

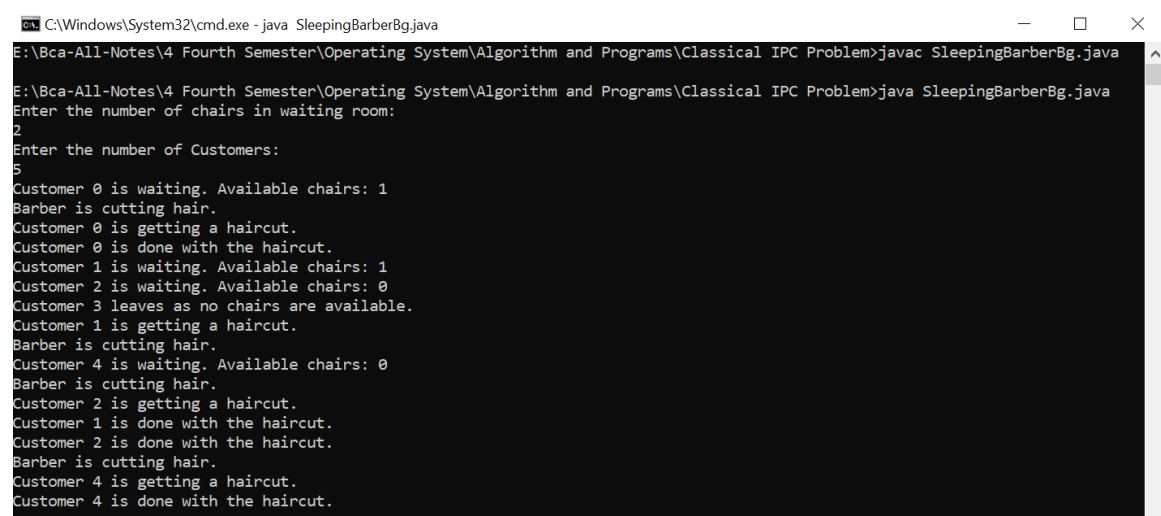
        } catch (InterruptedException e) {

e.printStackTrace();}

    }

}

```



```

C:\Windows\System32\cmd.exe - java SleepingBarberBg.java
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\Classical IPC Problem>javac SleepingBarberBg.java
E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\Classical IPC Problem>java SleepingBarberBg.java
Enter the number of chairs in waiting room:
2
Enter the number of Customers:
5
Customer 0 is waiting. Available chairs: 1
Barber is cutting hair.
Customer 0 is getting a haircut.
Customer 0 is done with the haircut.
Customer 1 is waiting. Available chairs: 1
Customer 2 is waiting. Available chairs: 0
Customer 3 leaves as no chairs are available.
Customer 1 is getting a haircut.
Barber is cutting hair.
Customer 4 is waiting. Available chairs: 0
Barber is cutting hair.
Customer 2 is getting a haircut.
Customer 1 is done with the haircut.
Customer 2 is done with the haircut.
Barber is cutting hair.
Customer 4 is getting a haircut.
Customer 4 is done with the haircut.

```

1. WAP to implement producer-consumer problem.

```
import java.util.Scanner;

public class ProducerConsumerBasic {

    public static void main(String[] args) {

        int buffer_Aakar = 5;

        int[] buffer = new int[buffer_Aakar];

        int vitra = 0, bira = 0;

        int choice = 0;

        Scanner hehe = new Scanner(System.in);

        while (choice != 3) {

            System.out.println("\n1. Producer\t2. Consumer\t3. Exit\nEnter your choice:");

            choice = hehe.nextInt();

            switch (choice) {

                case 1:

                    if ((vitra + 1) % buffer_Aakar == bira) {

                        System.out.println("Buffer Voriyo (Buffer Full)...");

                    } else {

                        System.out.print("Enter data to produce: ");

                        buffer[vitra] = hehe.nextInt();

                        vitra = (vitra + 1) % buffer_Aakar;

                    }

                    break;

                case 2:

                    if (bira == vitra) {

                        System.out.println("Buffer khali vayo (Buffer Empty)...");

                    } else {

                        int consumedData = buffer[bira];

                        System.out.println("Consumed data: " + consumedData);

                        bira = (bira + 1) % buffer_Aakar;

                    }

                    break;

            }

        }

    }

}
```

```

case 3:

System.out.println("Niklim Niklim...");

break;

default:

System.out.println("Invalid choice, try again.");

}

}

hehe.close();

}

}

```

```

C:\Windows\System32\cmd.exe
1 error

E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\Classical IPC Problem>javac ProducerConsumerBasic.java

E:\Bca-All-Notes\4 Fourth Semester\Operating System\Algorithm and Programs\Classical IPC Problem>java ProducerConsumerBasic.java

1. Producer    2. Consumer    3. Exit
Enter your choice:
1
Enter data to produce: 23

1. Producer    2. Consumer    3. Exit
Enter your choice:
2
Consumed data: 23

1. Producer    2. Consumer    3. Exit
Enter your choice:
2
Buffer khali vayo (Buffer Empty)...

1. Producer    2. Consumer    3. Exit
Enter your choice:
3
Niklim Niklim...

```

Process Creation

1. WAP to create Parent or child process.

```
#include <stdio.h>

#include <unistd.h>

int main() {

    pid_t pid = fork();

    if (pid == 0) {

        // Child process

        printf("Child Process\n");

        printf("Child PID: %d\n", getpid());

        printf("Parent PID: %d\n", getppid());

    }

    else if (pid > 0) {

        // Parent process

        printf("Parent Process\n");

        printf("Parent PID: %d\n", getpid());

    }

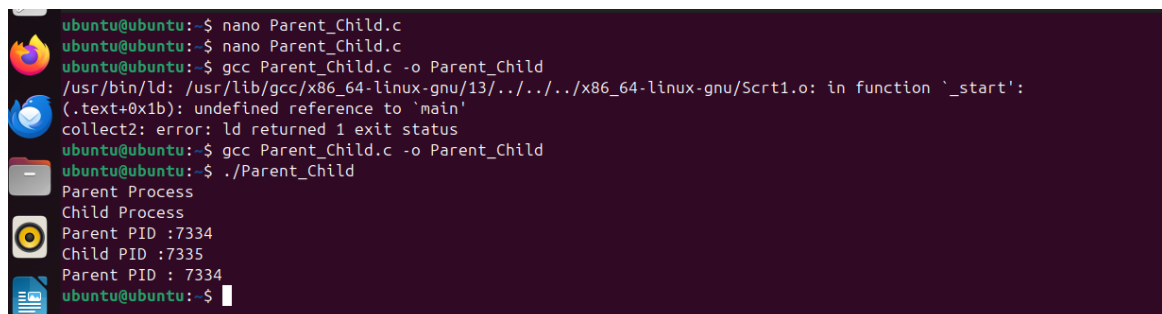
    else {

        printf("Fork failed\n");

    }

    return 0;

}
```



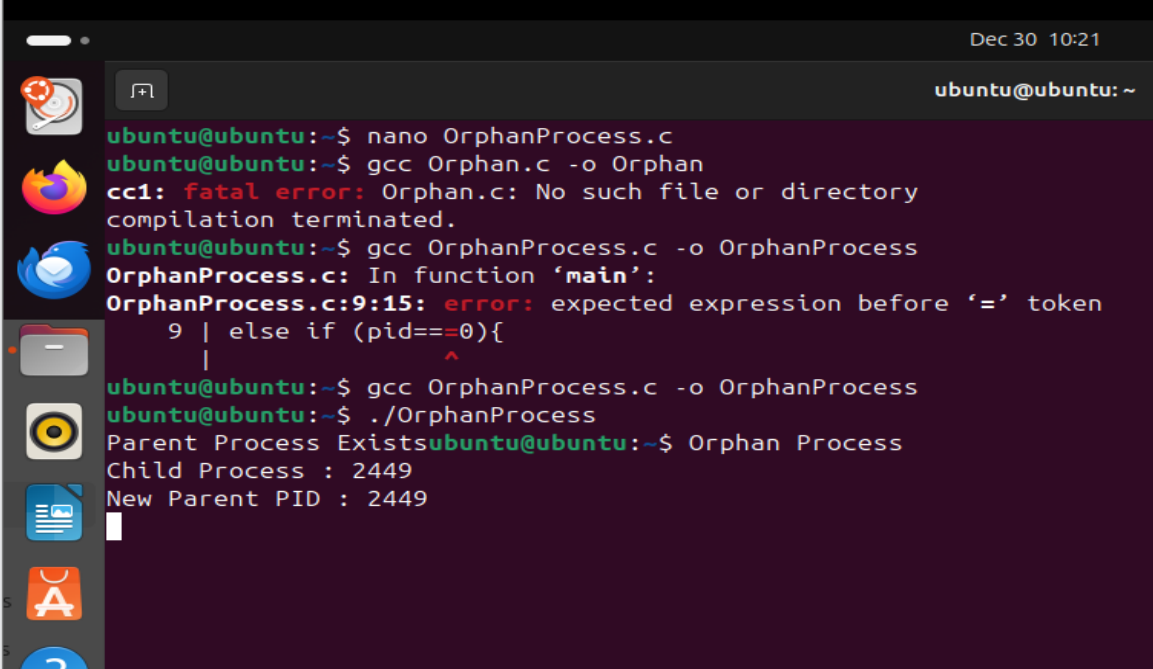
```
ubuntu@ubuntu:~$ nano Parent_Child.c
ubuntu@ubuntu:~$ nano Parent_Child.c
ubuntu@ubuntu:~$ gcc Parent_Child.c -o Parent_Child
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/13/../../../../x86_64-linux-gnu/Scrt1.o: in function '_start':
(.text+0x1b): undefined reference to 'main'
collect2: error: ld returned 1 exit status
ubuntu@ubuntu:~$ gcc Parent_Child.c -o Parent_Child
ubuntu@ubuntu:~$ ./Parent_Child
Parent Process
Child Process
Parent PID :7334
Child PID :7335
Parent PID : 7334
ubuntu@ubuntu:~$
```


2. WAP to create orphan process.

```
#include <stdio.h>

#include <unistd.h>

int main() {
    pid_t pid = fork();
    if (pid > 0) {
        // Parent process exits early
        printf("Parent Process Exists\n");
        return 0;
    }
    else if (pid == 0) {
        sleep(5);
        printf("Orphan Process\n");
        printf("Child PID: %d\n", getpid());
        printf("New Parent PID (init/systemd): %d\n", getppid());
    }
    return 0;
}
```



The screenshot shows a terminal window with the following commands and output:

```
ubuntu@ubuntu:~$ nano OrphanProcess.c
ubuntu@ubuntu:~$ gcc Orphan.c -o Orphan
cc1: fatal error: Orphan.c: No such file or directory
compilation terminated.
ubuntu@ubuntu:~$ gcc OrphanProcess.c -o OrphanProcess
OrphanProcess.c: In function 'main':
OrphanProcess.c:9:15: error: expected expression before '=' token
  9 | else if (pid==0){
    |             ^
ubuntu@ubuntu:~$ gcc OrphanProcess.c -o OrphanProcess
ubuntu@ubuntu:~$ ./OrphanProcess
Parent Process Exists
Orphan Process
Child Process : 2449
New Parent PID : 2449
```

3. WAP to create a demon Process

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/stat.h>

int main() {

    pid_t pid = fork();

    if (pid < 0) {

        exit(EXIT_FAILURE);

    }

    if (pid > 0) {

        exit(EXIT_SUCCESS);

    }

    if (setsid() < 0) {

        exit(EXIT_FAILURE);

    }

    chdir("/");

    close(STDIN_FILENO);

    close(STDOUT_FILENO);

    close(STDERR_FILENO);

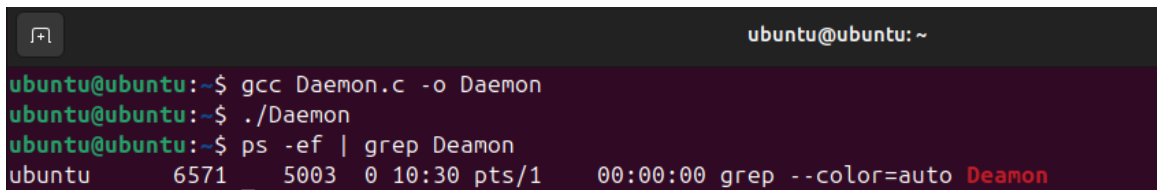
    while (1) {

        sleep(10);

    }

    return 0;

}
```



```
ubuntu@ubuntu: ~
ubuntu@ubuntu:~$ gcc Daemon.c -o Daemon
ubuntu@ubuntu:~$ ./Daemon
ubuntu@ubuntu:~$ ps -ef | grep Daemon
ubuntu      6571      5003  0 10:30 pts/1    00:00:00 grep --color=auto Daemon
```